

NM 文字题合集 V2

Chapter 0: 收敛分析 Convergence Analysis

Order / 阶数	Error ratio when step halved / 步长减半时误差比值	Meaning / 含义
1st order	≈ 2.0 / 误差减半	Forward/Backward Euler, Forward Difference
2nd order	≈ 4.0 / 误差变 1/4	Central Difference, Heun, Midpoint, Trapezoid
4th order	≈ 16.0 / 误差变 1/16	Simpson's, 4th-order Central Difference
6th order	≈ 64.0 / 误差变 1/64	Weddle's rule

Why can error stop decreasing when Δx is very small? / 为什么 Δx 很小时误差不再下降?

When Δx becomes very small, for example below 10^{-8} , round-off errors from finite-precision arithmetic can dominate over truncation errors. Therefore, the total error may stop decreasing, flatten out, or even increase. This should be mentioned when discussing the log-log error plot.

当 Δx 极小，例如小于 10^{-8} 时，有限精度计算中的 round-off errors 可能超过 truncation errors。因此，总误差可能不再下降，而是 flatten out，甚至 increase。在讨论 log-log error plot 时需要提到这一点。

Chapter 1: Interpolation & Curve Fitting 插值与曲线拟合

Interpolation vs Extrapolation vs Curve Fitting / 插值、外推与曲线拟合的区别

Interpolation constructs a function that passes exactly through all known data points and estimates values inside the data range. Extrapolation extends the function outside the data range, but accuracy usually drops quickly as the distance from the data increases. Curve fitting finds an approximating function that captures the overall trend without necessarily passing through every point; it minimises SSE and is preferred for noisy data.

Interpolation 构造一个精确通过所有已知 data points 的函数，用来估计数据范围内的值。Extrapolation 把这个函数延伸到数据范围之外，但离已知数据越远，准确性通常下降越快。Curve fitting 寻找一个近似函数来捕捉整体 trend，不一定通过每个点；它最小化 SSE，更适合 noisy data。

How many degrees are needed to fit n points exactly? / 精确拟合 n 个点需要几次多项式？

For n distinct data points, there exists a unique polynomial of degree at most $n - 1$ that interpolates them exactly. A degree- $(n - 1)$ polynomial has n unknown coefficients, and n data points provide n constraint equations.

对于 n 个不同的 data points，存在唯一一个次数最多为 $n - 1$ 的 polynomial 可以精确插值这些点。degree- $(n - 1)$ polynomial 有 n 个 unknown coefficients，而 n 个 data points 提供 n 个 constraint equations。

What does polyfit minimise? / polyfit 会将什么值最小化？

`polyfit` minimises the sum of squared errors, or SSE, between the fitted polynomial and the data points. This is closely related to RMS or RMSE, because both measure the size of the fitting error. A smaller SSE means the fitted curve is closer to the given data points.

`polyfit` 最小化 fitted polynomial 和 data points 之间的 sum of squared errors，也就是 SSE。它和 RMS / RMSE 密切相关，因为它们都衡量 fitting error 的大小。SSE 越小，说明 fitted curve 越贴近给定数据点。

Why might curve-fitting be preferred over interpolation for noisy data? / 噪声数据为什么优先选拟合？

Interpolation forces the polynomial to pass through every noisy data point, so it fits the noise as well as the underlying signal. This can produce large oscillations between points and poor generalisation. Curve fitting with a lower-degree polynomial captures the true trend while leaving random noise unexplained, so it often represents the true function better.

Interpolation 强制 polynomial 通过每一个 noisy data point，所以它会同时拟合 noise 和 underlying signal。这可能导致数据点之间出现明显 oscillations，并且 generalisation 较差。低次 curve fitting 可以捕捉 true trend，同时忽略 random noise，因此通常更能代表 true function。

Is a 10th-order polynomial always better? / 10 次多项式一定更好吗？

No. Although a 10th-order polynomial may give a lower SSE on the given data points, it is likely to overfit the data. A high-degree polynomial can oscillate strongly between points and give

unreliable extrapolation outside the data range. A lower-degree model is often better because it captures the true trend without fitting the noise too closely.

不一定。虽然 10th-order polynomial 可能在给定 data points 上有更低的 SSE，但它很可能出现 overfitting。高次 polynomial 可能在点之间剧烈 oscillate，并且在数据范围外的 extrapolation 不可靠。较低次数的模型通常更好，因为它能捕捉 true trend，而不是过度拟合 noise。

What is Runge's Phenomenon? / 什么是 Runge 振荡现象?

Runge's Phenomenon is the oscillation that can appear near the edges of an interval when using high-degree polynomial interpolation, even if the original function is smooth. A classic example is $f(x) = 1 / (1 + 25x^2)$ on $[-1, 1]$. With a high-degree polynomial, the fit can become poor near the boundaries, so piecewise low-degree interpolation or composite methods are usually preferred.

Runge's Phenomenon 是指：即使 original function 本身很 smooth，使用 high-degree polynomial interpolation 时，区间边缘附近仍可能出现明显 oscillations。经典例子是 $[-1, 1]$ 上的 $f(x) = 1 / (1 + 25x^2)$ 。高次 polynomial 在边界附近可能拟合很差，因此通常更倾向于 piecewise low-degree interpolation 或 composite methods。

How should we generate and plot noisy data? / 如何生成并画出 noisy data?

To generate noisy data, first choose uniformly spaced x values over the required interval, then evaluate the true function at these points and add a small random noise term. For example, the noise-free function may be $g(x) = \cos(2\pi x)$, and the noisy data are produced by adding random values with small standard deviation. Plotting both the noisy points and the original function helps show the difference between measured data and the underlying true signal.

生成 noisy data 时，先在给定区间内选 uniformly spaced x values，然后在这些点上计算 true function，再加一个 small random noise term。例如 noise-free function 可以是 $g(x) = \cos(2\pi x)$ ，noisy data 就是在它的基础上加入小标准差的随机扰动。把 noisy points 和 original function 同时画出来，可以看出 measured data 和 underlying true signal 的区别。

How does the Lagrange interpolant behave between noisy data points? / Lagrange 插值在噪声点之间会有什么表现?

The Lagrange interpolant passes exactly through every noisy data point, so its error at those data points is zero. However, this also means it fits the noise, not only the underlying function. Between data points, the polynomial may oscillate and move away from the true noise-free function, especially when the degree is high or the data are noisy.

Lagrange interpolant 会精确通过每一个 noisy data point，所以在这些数据点上的误差是 zero。但这也说明它不仅拟合 underlying function，还拟合了 noise。在 data points 之间，polynomial 可能出现

oscillations, 并偏离 true noise-free function, 尤其在 degree 较高或数据带噪声时更明显。

How do we compare MSE/RMSE against noisy data and the true function? / 如何分别对 noisy data 和 true function 计算 MSE/RMSE?

MSE or RMSE can be calculated in two different ways. Error against the noisy data points measures how well the approximation fits the observations used for fitting. Error against the true function, evaluated on many smooth plotting points, measures how well the approximation recovers the underlying signal. An interpolant may have zero error against noisy data, but still have a larger error against the true function because it also fits the noise.

MSE 或 RMSE 可以用两种方式计算。Against noisy data points 的误差衡量 approximation 多贴合用于拟合的观测数据。Against true function 的误差通常在很多 smooth plotting points 上计算, 用来衡量 approximation 多接近 underlying signal。Interpolant 对 noisy data 的误差可能是 zero, 但对 true function 的误差仍可能较大, 因为它也拟合了 noise。

Why can a higher-degree least-squares fit reduce data misfit but still be worse? / 为什么高阶 least-squares fit 降低 data misfit 但不一定更好?

Increasing the polynomial degree gives the model more flexibility, so it often reduces the misfit to the given data points. However, this does not always mean the model is better. A high-degree polynomial may begin to follow random noise rather than the true trend, giving poor behaviour between points or outside the data range. This is overfitting.

提高 polynomial degree 会让模型更 flexible, 所以通常能降低对给定 data points 的 misfit。但这不一定说明模型更好。高阶 polynomial 可能开始追随 random noise, 而不是 true trend, 导致点之间或数据范围外的表现变差。这就是 overfitting。

What is the difference between polynomial and non-polynomial functions? / polynomial 和 non-polynomial function 有什么区别?

A polynomial function can be written as a finite weighted sum of powers of x , such as $f(x) = \beta x + \alpha$ or $f(x) = \gamma x^2 + \beta x + \alpha$. A non-polynomial function cannot be written exactly as a finite polynomial, for example $g(x) = \sin(x)$ or $g(x) = \exp(x)$. Polynomial interpolation can exactly recover a polynomial if enough suitable sample points are used, but it can only approximate a non-polynomial function.

Polynomial function 可以写成有限个 x 的幂次加权和, 例如 $f(x) = \beta x + \alpha$ 或 $f(x) = \gamma x^2 + \beta x + \alpha$ 。Non-polynomial function 不能被有限次数 polynomial 精确表示, 例如 $g(x) = \sin(x)$ 或 $g(x) = \exp(x)$ 。Polynomial interpolation 如果有足够合适的 sample points, 可以精确恢复 polynomial, 但对 non-polynomial function 只能近似。

What does sampling a continuous function mean? / sampling 一个连续函数是什么意思?

Sampling means evaluating a continuous function at a finite number of selected x locations. This produces discrete data points (x, y) that represent the original continuous relationship only at those locations. Numerical methods then use these discrete samples to build an interpolant or fitted curve, but information between sample points must be inferred.

Sampling 是指在有限个选定的 x locations 上计算 continuous function 的值。这样会得到 discrete data points (x, y) ，它们只在这些位置代表原来的连续关系。Numerical methods 会用这些离散样本建立 interpolant 或 fitted curve，但 sample points 之间的信息需要推断。

Why can exact fitting of discrete points still fail to match the underlying function? / 为什么精确拟合离散点仍可能不匹配 underlying function?

Exact fitting only guarantees that the interpolant or fitted polynomial matches the sampled data points. It does not guarantee that the curve matches the original continuous function between those points. If the sample size N is too small, important features such as curvature, oscillations, or rapid variation may be missed, so the underlying function may be poorly represented.

Exact fitting 只保证 interpolant 或 fitted polynomial 通过 sampled data points，并不保证它在这些点之间也匹配 original continuous function。如果 sample size N 太小，curvature、oscillations 或 rapid variation 等重要特征可能被漏掉，所以 underlying function 可能被表示得很差。

Why can Lagrange interpolation exactly fit sampled data points? / 为什么 Lagrange interpolation 可以精确通过 sampled data points?

For N distinct sample points, the Lagrange interpolating polynomial has degree at most $N - 1$ and is constructed to pass exactly through every data point. Therefore, the interpolation error at the sampled points is zero. This exact fit applies to data sampled from both polynomial and non-polynomial functions, but it does not guarantee exact matching between the sample points.

对于 N 个不同的 sample points，Lagrange interpolating polynomial 的次数最多为 $N - 1$ ，并且构造时就要求它精确通过每个 data point。因此，在 sampled points 上的 interpolation error 是 zero。这个 exact fit 对来自 polynomial 和 non-polynomial functions 的数据都成立，但不保证在 sample points 之间也完全正确。

Why can a high-degree least-squares polynomial exactly fit discrete data? / 为什么高阶 least-squares polynomial 也可以精确拟合离散数据?

Least-squares fitting does not always pass through every point, but if the polynomial degree is high enough, it can have enough coefficients to satisfy all data-point constraints. For N distinct points, a degree $N - 1$ polynomial can exactly interpolate the data, so the least-squares misfit can become zero or nearly zero. This exact fit is to the discrete data, not necessarily to the original continuous function.

Least-squares fitting 不一定通过每个点，但如果 polynomial degree 足够高，它就有足够多的 coefficients 来满足所有 data-point constraints。对于 N 个不同点，degree $N - 1$ polynomial 可以精确插值这些数据，所以 least-squares misfit 可以变成 zero 或接近 zero。这里精确拟合的是 discrete data，不一定是原来的 continuous function。

Why can too few sample points fail to represent the underlying function? / 为什么 sample points 太少不能代表 underlying function?

If N is too small, the sampled data may not capture the shape, curvature, or oscillations of the underlying function. The interpolant can still pass exactly through the sampled points, but many different continuous functions could pass through the same few points. Therefore, exact fit to sparse samples does not mean exact recovery of the original function.

如果 N 太小，sampled data 可能无法捕捉 underlying function 的 shape、curvature 或 oscillations。Interpolant 仍然可以精确通过这些 sample points，但很多不同的 continuous functions 都可能通过同样少的点。因此，对稀疏样本的 exact fit 不等于精确恢复 original function。

Why can interpolation exactly recover a polynomial but not a non-polynomial function? / 为什么 interpolation 可以精确恢复 polynomial 但不能精确恢复 non-polynomial?

If the true function is a polynomial of degree m , then using at least $m + 1$ suitable distinct sample points allows polynomial interpolation to recover it exactly. This is because the interpolant has enough information to identify all coefficients of the polynomial. For a non-polynomial function such as $\sin(x)$ or $\exp(x)$, no finite-degree polynomial can match it exactly everywhere over an interval.

如果 true function 是 degree m 的 polynomial，那么使用至少 $m + 1$ 个合适且不同的 sample points，就可以用 polynomial interpolation 精确恢复它。因为 interpolant 有足够信息确定这个 polynomial 的全部 coefficients。对于 $\sin(x)$ 或 $\exp(x)$ 这类 non-polynomial function，没有任何 finite-degree polynomial 能在整个区间上完全匹配它。

Would the interpolation error always decrease as N increases? / interpolation error 是否一定随 N 增加而下降?

Not always. Increasing N usually gives more information about the underlying function, so the error may decrease. However, high-degree polynomial interpolation can also introduce oscillations, especially near interval edges or with poorly chosen sample points. Therefore, the error against a non-polynomial function may not decrease monotonically as N increases.

不一定。增加 N 通常提供更多 underlying function 信息，所以 error 可能下降。但 high-degree polynomial interpolation 也可能引入 oscillations，尤其在区间边缘或 sample points 选择不好时。因此，对 non-polynomial function 的误差不一定随 N 增加而单调下降。

When might the actual interpolating polynomial degree be lower than $n - 1$? / 什么时候实际插值多项式次数会低于 $n - 1$?

For n distinct data points, a polynomial of degree at most $n - 1$ can interpolate them exactly. However, the actual polynomial degree may be lower if the data points already lie on a simpler function. For example, five points may be interpolated by a degree 4 polynomial in theory, but if they all lie on a straight line, the actual interpolating polynomial is only degree 1.

对于 n 个不同的 data points，次数最多为 $n - 1$ 的 polynomial 可以精确插值它们。但实际 polynomial degree 可能更低，如果这些点本来就落在更简单的函数上。例如 5 个点理论上可以用 degree 4 polynomial 插值，但如果它们都在一条 straight line 上，实际插值多项式只有 degree 1。

How does least-squares error change as polynomial degree increases? / least-squares error 如何随 polynomial degree 增加而变化?

In least-squares curve fitting, increasing the polynomial degree gives the model more flexibility. Therefore, the fitting error to the data points usually decreases or stays the same, because the higher-degree model can reproduce everything the lower-degree model can do. However, a smaller data error does not always mean a better model, because high-degree models may overfit noise.

在 least-squares curve fitting 中，提高 polynomial degree 会让模型更 flexible。因此，对 data points 的 fitting error 通常会下降或保持不变，因为高阶模型可以做到低阶模型能做到的事情。但 data error 更小不一定说明模型更好，因为 high-degree models 可能 overfit noise。

When is a linear fit good enough? / 什么时候 linear fit 已经足够好?

A linear fit is good enough when the data cloud has an approximately linear trend and higher-degree polynomials only reduce the least-squares error by a very small amount. In this case, the remaining error is mostly caused by scatter or randomness rather than missing curvature. A linear model is then preferred because it is simpler, easier to interpret, and avoids unnecessary overfitting.

当 data cloud 大致呈 linear trend，并且更高阶 polynomial 只能让 least-squares error 减少很少时，linear fit 就已经足够好。这时剩余 error 主要来自 scatter 或 randomness，而不是缺少 curvature。Linear model 更简单、更容易解释，也能避免不必要的 overfitting。

When is a quadratic fit better than a linear fit? / 什么时候 quadratic fit 比 linear fit 更好?

A quadratic fit is better when the data cloud shows clear curvature that a straight line cannot capture. In this case, moving from a linear to a quadratic polynomial should significantly reduce the least-squares error. If increasing the degree beyond quadratic gives little extra improvement, then the quadratic model is a good balance between accuracy and simplicity.

当 data cloud 有明显 curvature，而 straight line 无法捕捉这种弯曲趋势时，quadratic fit 会更好。这种情况下，从 linear polynomial 增加到 quadratic polynomial 应该明显降低 least-squares error。如果继续提高 degree 只带来很小改进，那么 quadratic model 就是 accuracy 和 simplicity 之间比较好的平衡。

Why is the simplest adequate polynomial often preferred? / 为什么通常优先选择足够好的低阶 polynomial?

The simplest adequate polynomial is often preferred because it captures the main trend without adding unnecessary flexibility. Higher-degree polynomials may slightly reduce the fitting error, but they can also fit random scatter and behave poorly between data points. A good model should balance low error, smooth behaviour, and interpretability.

通常优先选择足够好的低阶 polynomial，因为它能捕捉 main trend，同时不增加不必要的 flexibility。高阶 polynomial 可能略微降低 fitting error，但也可能拟合 random scatter，并在 data points 之间表现很差。好的模型需要平衡 low error、smooth behaviour 和 interpretability。

When can interpolation be unsafe or unreliable? / 什么时候 interpolation 可能不安全或不可靠?

Interpolation is usually reasonable only inside the range of the known data. It can become unreliable if the data points are very sparse, noisy, or unevenly distributed, because the interpolant may behave poorly between points. High-degree interpolation can also create oscillations, so passing exactly through the data does not always mean the curve is physically meaningful.

Interpolation 通常只在 known data 的范围内部比较合理。如果 data points 很 sparse、noisy 或分布不均匀，interpolant 在点之间可能表现很差，因此结果不可靠。High-degree interpolation 也可能产生 oscillations，所以精确通过数据点不一定代表曲线有物理意义。

Why can extrapolation be dangerous? / 为什么 extrapolation 可能危险?

Extrapolation estimates values outside the range of the data, where there is no direct information to constrain the function. Even if a fitted curve works well inside the data range, it may grow, bend, or oscillate incorrectly outside that range. Therefore, extrapolation is often much less reliable than interpolation and should be used with caution.

Extrapolation 是在 data range 外估计数值，而这些区域没有直接数据来约束函数。即使 fitted curve 在数据范围内表现很好，在范围外也可能错误增长、弯曲或振荡。因此 extrapolation 通常比 interpolation 更不可靠，需要谨慎使用。

When can curve-fitting be inappropriate? / 什么时候 curve-fitting 可能不合适?

Curve-fitting can be inappropriate if the chosen model form does not match the data trend. For example, using a linear fit for strongly curved data may underfit the pattern, while using a very high-degree polynomial for scattered data may overfit random noise. A good fit should balance data error, simplicity, and sensible behaviour between and beyond the data points.

如果选择的 model form 和 data trend 不匹配，curve-fitting 就可能不合适。例如，对明显弯曲的数据使用 linear fit 会 underfit；对 scattered data 使用 very high-degree polynomial 可能 overfit random noise。好的拟合需要平衡 data error、simplicity，以及在数据点之间和之外的合理表现。

Can we always use interpolation, extrapolation and curve-fitting? / interpolation、extrapolation 和 curve-fitting 是否总能使用?

These methods can often be applied computationally, but they are not always meaningful. Interpolation requires enough suitable data points inside the range of interest. Extrapolation is risky because it predicts outside the data range. Curve-fitting depends strongly on choosing an appropriate model and avoiding underfitting or overfitting.

这些方法在计算上通常可以使用，但不一定总是有意义。Interpolation 需要在目标范围内有足够合适的的数据点。Extrapolation 风险较高，因为它预测的是 data range 之外的值。Curve-fitting 很依赖合适的 model choice，并且需要避免 underfitting 或 overfitting。

How do we derive a Lagrange polynomial by hand? / 如何手推 Lagrange polynomial?

To derive a Lagrange polynomial by hand, construct one basis polynomial for each data point. Each basis polynomial equals 1 at its own point and 0 at the other sample points. The interpolating polynomial is then the weighted sum of these basis polynomials, using the corresponding y values as weights.

手推 Lagrange polynomial 时，需要为每个 data point 构造一个 basis polynomial。每个 basis polynomial 在自己的点上等于 1，在其他 sample points 上等于 0。Interpolating polynomial 就是这些 basis polynomials 的加权和，权重是对应的 y values。

How does the number of data points affect the Lagrange polynomial degree? / data points 数量如何影响 Lagrange polynomial degree?

With two distinct data points, the Lagrange interpolant is at most linear. With three distinct data points, it is at most quadratic. In general, n distinct points give an interpolating polynomial of degree at most $n - 1$. The actual degree can be lower if the points lie on a simpler polynomial.

对于两个不同 data points，Lagrange interpolant 最多是 linear。对于三个不同 data points，最多是 quadratic。一般来说， n 个 distinct points 给出的 interpolating polynomial 次数最多是 $n - 1$ 。如果这些点本来落在更简单的 polynomial 上，实际次数可能更低。

How can four distinct points be interpolated by a quadratic? / 四个不同点为什么可以用 quadratic 插值?

Four distinct data points can be interpolated by a quadratic if all four points lie exactly on the same quadratic curve. Although four general points require a polynomial of degree at most 3, special data may have a lower-degree interpolant. For example, any four points sampled from $y = x^2$ can be represented exactly by a quadratic.

如果四个不同 data points 全部精确落在同一条 quadratic curve 上，就可以用 quadratic 插值。虽然一般四个点需要次数最多为 3 的 polynomial，但特殊数据可能有更低阶的 interpolant。例如，从 $y = x^2$ 上取的任意四个点都可以被 quadratic 精确表示。

Chapter 2: Numerical Differentiation 数值微分

Method / 方法	Ratio / 比值	Order / 阶数	Sentence to copy / 抄写
Forward	≈ 2.0	1st $O(h)$	"Forward difference is first-order $O(h)$."
Backward	≈ 2.0	1st $O(h)$	"Backward difference is first-order $O(h)$."
Central (2nd)	≈ 4.0	2nd $O(h^2)$	"Central difference is second-order $O(h^2)$."

Central (4th)	≈ 16.0	4th $O(h^4)$	"Fourth-order central difference is $O(h^4)$."
---------------	----------------	--------------	---

Why do we need to discretise? / 为什么需要离散化?

Computers can only store and operate on finite amounts of data. A continuous function such as $f(x)$ is defined at infinitely many points, which cannot be represented exactly in digital memory. Discretisation replaces the continuous domain with a finite mesh or grid of points, allowing us to compute numerical approximations using a finite number of operations.

计算机只能存储和处理有限数量的数据。像 $f(x)$ 这样的 continuous function 在无穷多个点上有定义，不能被 digital memory 精确表示。Discretisation 用有限个 mesh / grid points 代替连续区域，使我们能用有限次运算进行 numerical approximation。

Why is central difference more accurate than forward difference? / 为什么中心差分比前向差分更准确?

Forward difference has leading error term $\frac{h}{2} f''(x)$, so its error is $O(h)$ and it is first-order accurate. Central difference uses function values on both sides of x , so the first-order error terms cancel by symmetry. Its leading error term is $\frac{h^2}{6} f'''(x)$, so its error is $O(h^2)$ and it is second-order accurate.

Forward difference 的 leading error term 是 $\frac{h}{2} f''(x)$ ，所以 error 是 $O(h)$ ，属于 first-order accurate。Central difference 使用 x 两侧的 function values，因此 first-order error terms 会因为 symmetry 抵消。它的 leading error term 是 $\frac{h^2}{6} f'''(x)$ ，所以 error 是 $O(h^2)$ ，属于 second-order accurate。

Why can the error increase when Δx is very small? / 为什么 Δx 很小时误差反而会增加?

For moderate Δx , decreasing Δx reduces the truncation error, so the numerical derivative becomes more accurate. However, when Δx is extremely small, subtracting nearly equal function values can cause loss of significant digits. At this stage, round-off error dominates, so the total error may level off or increase instead of following the theoretical order of convergence.

在中等大小的 Δx 下，减小 Δx 会降低 truncation error，所以 numerical derivative 更准确。但当 Δx 极小时，两个非常接近的 function values 相减会造成 significant digits loss。此时 round-off error 占主导，所以总误差可能 flatten out 或 increase，而不是继续按 theoretical order of convergence 下降。

Why is numerical differentiation necessary? / 为什么需要 numerical differentiation?

Numerical differentiation is necessary when the exact analytical derivative is unavailable, too complicated, or when the function is only given through data points or computer code. In these cases, we approximate the derivative using nearby function values. Finite difference methods replace the limiting definition of the derivative with a computable formula using a small spacing h or Δx .

当 exact analytical derivative 不存在、太复杂，或者函数只通过 data points / computer code 给出时，就需要 numerical differentiation。这时我们用附近的 function values 来近似 derivative。Finite difference methods 用 small spacing h 或 Δx 下的可计算公式，代替 derivative 的极限定义。

Would central difference always outperform forward difference? / Central difference 是否一定永远优于 forward difference?

Central difference is usually more accurate because its leading truncation error is $O(h^2)$, while forward difference has leading error $O(h)$. However, central difference does not always outperform in every practical situation. Near a boundary, $f(x-h)$ may not be available, so forward difference may be the only option. When h is extremely small, round-off error can also dominate both methods.

Central difference 通常更准确，因为它的 leading truncation error 是 $O(h^2)$ ，而 forward difference 是 $O(h)$ 。但 central difference 不是所有实际情况都一定更好。比如在 boundary 附近， $f(x-h)$ 可能不可用，这时只能用 forward difference。当 h 极小时，round-off error 也可能同时主导两种方法。

How do Taylor expansions derive forward and central difference errors? / 如何用 Taylor expansion 推导 forward 和 central difference 的误差?

Taylor expansion explains the accuracy of finite difference formulae. For forward difference, expanding $f(x+h)$ around x gives $f(x+h) = f(x) + h f'(x) + \frac{h^2}{2} f''(x) + \dots$. After rearranging, $[f(x+h) - f(x)]/h = f'(x) + \frac{h}{2} f''(x) + O(h^2)$, so the leading error term is $\frac{h}{2} f''(x)$ and the method is first-order accurate, with error $O(h)$.

Taylor expansion 可以解释 finite difference formulae 的精度。对于 forward difference，把 $f(x+h)$ 在 x 附近展开，得到 $f(x+h) = f(x) + h f'(x) + \frac{h^2}{2} f''(x) + \dots$ 。整理后， $[f(x+h) - f(x)]/h = f'(x) + \frac{h}{2} f''(x) + O(h^2)$ ，所以 leading error term 是 $\frac{h}{2} f''(x)$ ，该方法是 first-order accurate，误差为 $O(h)$ 。

How should we interpret a log-log convergence plot? / 如何解释 log-log convergence plot?

In a log-log error plot, the slope of the line represents the observed order of accuracy. If the forward difference error behaves like $O(h)$, the slope should be close to 1. If the central difference error behaves like $O(h^2)$, the slope should be close to 2. For very small h , the curve may flatten out or rise because round-off error begins to dominate over truncation error.

在 log-log error plot 中, 线的 slope 表示 observed order of accuracy。如果 forward difference 的误差表现为 $O(h)$, slope 应接近 1。如果 central difference 的误差表现为 $O(h^2)$, slope 应接近 2。当 h 非常小时, 曲线可能 flatten out 或上升, 因为 round-off error 开始超过 truncation error。

What is the backward finite difference approximation? / 什么是 backward finite difference?

The backward finite difference uses the current point and one point behind it to approximate the derivative. Its formula is $f'(x) \approx [f(x) - f(x-h)] / h$. Like the forward difference, it has leading truncation error $O(h)$, so it is first-order accurate. It is useful near a right boundary where $f(x+h)$ is not available.

Backward finite difference 使用当前点和它后方的一个点来近似 derivative。公式是 $f'(x) \approx [f(x) - f(x-h)] / h$ 。和 forward difference 一样, 它的 leading truncation error 是 $O(h)$, 所以是 first-order accurate。它适合在右边界附近使用, 因为那里可能没有 $f(x+h)$ 。

How do forward, backward and central differences compare? / forward、backward 和 central difference 如何比较?

Forward difference uses $f(x)$ and $f(x+h)$, backward difference uses $f(x-h)$ and $f(x)$, and central difference uses $f(x-h)$ and $f(x+h)$. Forward and backward differences are one-sided and first-order accurate. Central difference is two-sided and usually second-order accurate because its symmetric structure cancels the first-order error term.

Forward difference 使用 $f(x)$ 和 $f(x+h)$, backward difference 使用 $f(x-h)$ 和 $f(x)$, central difference 使用 $f(x-h)$ 和 $f(x+h)$ 。Forward 和 backward 是 one-sided methods, 通常是 first-order accurate。Central difference 是 two-sided method, 通常是 second-order accurate, 因为它的对称结构会抵消 first-order error term。

How does Taylor analysis work for backward difference? / Taylor analysis 如何推导 backward difference?

For backward difference, expand $f(x-h)$ about x : $f(x-h) = f(x) - h f'(x) + h^2 f''(x)/2 - \dots$. Rearranging gives $[f(x) - f(x-h)] / h = f'(x) - h f''(x)/2 + \dots$. Therefore, the leading error is proportional to h , so the backward difference has accuracy $O(h)$.

对 backward difference, 把 $f(x-h)$ 在 x 附近展开: $f(x-h) = f(x) - h f'(x) + \frac{h^2}{2} f''(x) - \dots$ 。整理后得到 $[f(x) - f(x-h)] / h = f'(x) - \frac{h}{2} f''(x) + \dots$ 。因此 leading error 和 h 成正比, 所以 backward difference 的精度是 $O(h)$ 。

How does convergence analysis confirm the Taylor series error prediction? / convergence analysis 如何验证 Taylor series 的误差预测?

Taylor analysis predicts how the error should scale with mesh spacing h . Convergence analysis checks this prediction by computing numerical errors for decreasing h values and plotting them on a log-log graph. If forward and backward differences show slope close to 1, and central difference shows slope close to 2, the numerical results agree with the Taylor error analysis.

Taylor analysis 会预测 error 如何随 mesh spacing h 变化。Convergence analysis 通过对不断减小的 h 计算 numerical errors, 并在 log-log graph 上画出来, 来检查这个预测。如果 forward 和 backward differences 的 slope 接近 1, central difference 的 slope 接近 2, 就说明 numerical results 和 Taylor error analysis 一致。

How do we check finite difference implementations using a known derivative? / 如何用已知导数检查 finite difference 代码?

To check a finite difference implementation, choose a function whose exact derivative is known, such as $f(x) = \cos(x)$. At $x = \pi/2$, the exact derivative is $f'(x) = -\sin(x) = -1$. By computing numerical derivatives for decreasing h and comparing with -1 , we can check whether the error decreases at the expected order.

检查 finite difference 代码时, 可以选择一个 exact derivative 已知的函数, 例如 $f(x) = \cos(x)$ 。在 $x = \pi/2$ 处, 精确导数是 $f'(x) = -\sin(x) = -1$ 。通过对不断减小的 h 计算 numerical derivatives, 并和 -1 比较, 就能检查 error 是否按预期阶数下降。

What is the fourth-order central difference approximation? / 什么是 fourth-order central difference?

The fourth-order central difference approximation uses two points on each side of x : $f(x+2h)$, $f(x+h)$, $f(x-h)$, and $f(x-2h)$. Its formula is $f'(x) \approx [-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)] / (12h)$. It is more accurate for smooth functions because its leading truncation error is $O(h^4)$.

Fourth-order central difference 使用 x 两侧各两个点: $f(x+2h)$ 、 $f(x+h)$ 、 $f(x-h)$ 和 $f(x-2h)$ 。公式是 $f'(x) \approx [-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)] / (12h)$ 。对于 smooth functions, 它更准确, 因为 leading truncation error 是 $O(h^4)$ 。

How do first-, second- and fourth-order finite differences compare? / 一阶、二阶、四阶差分如何比较?

A first-order method has error proportional to h , a second-order method has error proportional to h^2 , and a fourth-order method has error proportional to h^4 . Therefore, for suitable h , higher-order methods usually reduce error faster as the mesh is refined. However, if h becomes too small, round-off error can still dominate and reduce the expected benefit.

First-order method 的误差和 h 成正比, second-order method 的误差和 h^2 成正比, fourth-order method 的误差和 h^4 成正比。因此在合适的 h 范围内, 阶数越高, mesh refinement 时误差下降越快。但如果 h 太小, round-off error 仍然可能主导, 从而削弱理论优势。

What are error decrease factors when halving Δx ? / Δx 减半时 error decrease factor 是什么?

If a method has error $O(\Delta x^p)$, then halving Δx should reduce the error by about 2^p . Therefore, a first-order method should improve by a factor of 2, a second-order method by a factor of 4, and a fourth-order method by a factor of 16. These factors are useful for checking whether the implementation matches the theoretical order.

如果某方法的误差是 $O(\Delta x^p)$, 那么把 Δx 减半时, 误差大约会减少 2^p 倍。因此 first-order method 的 error decrease factor 约为 2, second-order 是 4, fourth-order 是 16。这些 factors 可以用来检查 implementation 是否符合 theoretical order。

What slope should we expect on a log-log convergence plot? / log-log convergence plot 上应该期待什么 slope?

On a log-log plot of error against Δx , the slope gives the order of accuracy p . Therefore, first-, second-, and fourth-order methods should have slopes close to 1, 2, and 4, respectively. The error decrease factors when halving Δx are 2, 4, and 16, but these are not the log-log slopes.

在 error 对 Δx 的 log-log plot 上, slope 表示 order of accuracy p 。因此 first-order、second-order 和 fourth-order methods 的 slope 应分别接近 1、2 和 4。当 Δx 减半时的 error decrease factors 是 2、4 和 16, 但这些不是 log-log slopes。

Why should we use the magnitude of error when taking logs? / 为什么取 log 时要用 error magnitude?

Logarithms are only defined for positive values, so we should use the absolute error or error magnitude when making a log-log convergence plot. The sign of the error only tells us whether the

approximation is above or below the exact value. For convergence, we care about the size of the error, not its sign.

Logarithm 只能对正数取值，所以画 log-log convergence plot 时应该使用 absolute error 或 error magnitude。Error 的正负只说明 approximation 在 exact value 上方还是下方。对于 convergence，我们关心的是 error size，而不是 sign。

How can Taylor series approximate the derivative directly? / Taylor series 如何直接近似 derivative?

Taylor series can approximate a smooth function near a chosen point, such as $t_0 = 0$, using polynomial terms. After writing the Taylor expansion for $f(t)$, we can differentiate the truncated polynomial to approximate $f'(t)$. Adding more Taylor terms usually improves the derivative approximation near the expansion point, but the approximation may become worse farther away.

Taylor series 可以在某个选定点附近，例如 $t_0 = 0$ ，用 polynomial terms 近似一个 smooth function。写出 $f(t)$ 的 Taylor expansion 后，可以直接对 truncated polynomial 求导来近似 $f'(t)$ 。加入更多 Taylor terms 通常会改善 expansion point 附近的 derivative approximation，但离展开点越远效果可能变差。

How is Taylor polynomial differentiation different from finite difference? / Taylor polynomial 求导和 finite difference 有什么区别?

Finite difference approximates the derivative using nearby function values and a small spacing Δt . Taylor polynomial differentiation first builds a polynomial approximation to the whole function, then differentiates that polynomial. Finite difference accuracy depends strongly on Δt , while Taylor approximation accuracy depends on the number of retained terms and the distance from the expansion point.

Finite difference 用附近的 function values 和 small spacing Δt 近似 derivative。Taylor polynomial differentiation 是先用 polynomial approximation 近似整个函数，再对这个 polynomial 求导。Finite difference 的精度强烈依赖 Δt ，而 Taylor approximation 的精度取决于保留的 terms 数量以及离 expansion point 的距离。

What happens when more Taylor terms are included? / 加入更多 Taylor terms 会发生什么?

Including more Taylor terms usually gives a better local approximation to the function and therefore a better derivative approximation. For $f(t) = \exp(t)$, every derivative is also $\exp(t)$, so the Taylor series about $t_0 = 0$ becomes a polynomial with increasing powers of t . Truncating after higher powers generally reduces the error near t_0 , but it is still an approximation.

加入更多 Taylor terms 通常会让函数的 local approximation 更好, 因此 derivative approximation 也更好。对于 $f(t) = \exp(t)$, 每阶导数仍是 $\exp(t)$, 所以在 $t_0 = 0$ 附近的 Taylor series 会变成包含更高次 t 的 polynomial。保留更高次项通常会降低 t_0 附近的 error, 但它仍然只是 approximation。

When should we use forward, backward, central and fourth-order differences? / forward、backward、central 和 fourth-order difference 分别什么时候用?

Forward difference is useful near a left boundary, where $f(x-h)$ is not available. Backward difference is useful near a right boundary, where $f(x+h)$ is not available. Central difference is preferred inside the domain because it uses information from both sides and is usually more accurate. Fourth-order central difference is even more accurate for smooth functions, but it needs values at $x \pm h$ and $x \pm 2h$.

Forward difference 适合在 left boundary 附近使用, 因为那里可能没有 $f(x-h)$ 。Backward difference 适合在 right boundary 附近使用, 因为那里可能没有 $f(x+h)$ 。Central difference 更适合在 domain 内部使用, 因为它利用两侧信息, 通常更准确。Fourth-order central difference 对 smooth functions 更精确, 但需要 $x \pm h$ 和 $x \pm 2h$ 处的函数值。

What are the limitations of finite difference methods? / finite difference methods 有什么限制?

Finite difference methods require the function to be evaluated at nearby points. They can fail or become difficult near boundaries if the required neighbouring points are outside the domain. They also assume the function is sufficiently smooth. If h is too large, truncation error is large; if h is too small, round-off error can dominate.

Finite difference methods 需要在附近 points 上计算 function value。如果在 boundary 附近需要的点超出 domain, 方法就会受限。它们也要求 function 足够 smooth。如果 h 太大, truncation error 会大; 如果 h 太小, round-off error 可能占主导。

What is the rule applicability of finite difference methods? / finite difference rules 的适用性是什么?

The basic applicability rule is: use one-sided differences near boundaries, use central differences in the interior, and use higher-order formulae only when the function is smooth and enough neighbouring points are available. Higher-order methods are not automatically better in all cases, because they require more points and can still suffer from round-off error.

基本适用原则是：boundary 附近用 one-sided differences，domain 内部优先用 central differences；只有当 function smooth 且有足够 neighbouring points 时，才适用用 higher-order formulae。Higher-order methods 不一定永远更好，因为它们需要更多点，并且仍可能受到 round-off error 影响。

Chapter 3: Numerical Integration 数值积分

Method / 方法	Ratio / 比值	Order / 阶数
Midpoint	≈ 4.0	2nd $O(h^2)$
Trapezoidal	≈ 4.0	2nd $O(h^2)$
Simpson's	≈ 16.0	4th $O(h^4)$
Weddle's	≈ 64.0	6th $O(h^6)$

Why use composite rules? / 为什么使用 Composite rules?

Composite rules split the whole interval into many smaller sub-intervals and apply a simple quadrature rule on each part. This avoids relying on one single high-degree polynomial over a large interval, which can behave poorly. Composite rules can also reuse shared interior points when the grid is refined, reducing redundant function evaluations.

Composite rules 把整个积分区间分成很多 smaller sub-intervals，然后在每一小段上使用简单的 quadrature rule。这样可以避免在大区间上依赖单个 high-degree polynomial，因为它可能表现很差。细化 grid 时，composite rules 也可以重复利用 shared interior points，从而减少 redundant function evaluations。

Why is $(4I_{h/2} - I_h)/3$ more accurate? / 为什么 Richardson 外推更准确?

If the leading error in the trapezoidal rule is $O(h^2)$, then we can write $I_h \approx I + ah^2$ and $I_{h/2} \approx I + ah^2 / 4$. Multiplying $I_{h/2}$ by 4 and subtracting I_h cancels the leading ah^2 error term. Therefore, $(4I_{h/2} - I_h) / 3$ gives a more accurate $O(h^4)$ estimate.

如果 trapezoidal rule 的 leading error 是 $O(h^2)$ ，那么可以写成 $I_h \approx I + ah^2$ 和 $I_{h/2} \approx I + ah^2 / 4$ 。把 $I_{h/2}$ 乘以 4 再减去 I_h ，可以抵消 leading ah^2 error term。因此 $(4I_{h/2} - I_h) / 3$ 会给出更高精度的 $O(h^4)$ 估计。

Why does Richardson extrapolation improve the trapezoidal rule? / 为什么 Richardson 外推能改进梯形法?

Richardson extrapolation improves the trapezoidal rule by cancelling its leading error term. Since the trapezoidal rule has error mainly proportional to h^2 , combining two approximations with step sizes h and $h/2$ removes this dominant part of the error. The result is a higher-order approximation with error $O(h^4)$.

Richardson extrapolation 通过抵消 leading error term 来改进 trapezoidal rule。因为 trapezoidal rule 的误差主要和 h^2 成正比，所以把步长为 h 和 $h/2$ 的两个近似结果组合起来，可以去掉这部分 dominant error。最终得到 error 为 $O(h^4)$ 的更高阶近似。

Why use non-uniform discretisation? / 为什么使用非均匀离散化?

Non-uniform discretisation is useful when a function changes much faster in some regions than others. More sub-intervals can be placed in rapidly varying regions, while fewer are used in smoother regions. This can achieve similar accuracy with fewer total function evaluations, and is often called adaptive or non-uniform quadrature.

Non-uniform discretisation 适合函数在某些区域变化明显更快的情况。变化快的区域可以放更多 sub-intervals，变化平缓的区域可以放更少 sub-intervals。这样可以用更少的 total function evaluations 达到类似精度，通常称为 adaptive 或 non-uniform quadrature。

How do we check an analytical integral using a trusted numerical method? / 如何用 trusted numerical method 检查解析积分?

After finding the analytical integral by hand, a trusted numerical method can be used as an independent check. For example, `scipy.integrate.quad` can provide a highly accurate numerical value for a smooth one-dimensional integral. If this numerical result agrees closely with the analytical value, it supports that the analytical calculation is correct.

手算得到 analytical integral 后，可以用 trusted numerical method 做独立检查。例如 `scipy.integrate.quad` 可以对 smooth one-dimensional integral 给出高精度 numerical value。如果这个 numerical result 和 analytical value 非常接近，就说明解析计算大概率正确。

What are the Midpoint, Trapezoid, and Simpson rules? / Midpoint、Trapezoid 和 Simpson rules 是什么?

Midpoint, Trapezoid, and Simpson rules are numerical quadrature methods for approximating definite integrals. The Midpoint rule uses the function value at the centre of each sub-interval. The Trapezoid rule approximates the function by a straight line between the two endpoints. Simpson's

rule uses a quadratic approximation over pairs of sub-intervals, so it is usually more accurate for smooth functions.

Midpoint、Trapezoid 和 Simpson rules 都是用来近似 definite integrals 的 numerical quadrature methods。Midpoint rule 使用每个 sub-interval 中点处的 function value。Trapezoid rule 用两个端点之间的 straight line 近似函数。Simpson's rule 在每两个 sub-interval 上使用 quadratic approximation, 所以对 smooth functions 通常更准确。

How do integration errors converge with number of sub-intervals N ? / 积分误差如何随 sub-interval number N 收敛?

As the number of sub-intervals N increases, the mesh spacing h decreases, so the numerical integration error usually decreases. On a log-log plot, Midpoint and Trapezoid rules normally show second-order convergence, while Simpson's rule normally shows fourth-order convergence. Since h is proportional to $1/N$, the error decreases roughly like N^{-2} for Midpoint and Trapezoid, and N^{-4} for Simpson.

随着 sub-interval number N 增加, mesh spacing h 变小, 所以 numerical integration error 通常下降。在 log-log plot 上, Midpoint 和 Trapezoid rules 通常表现为 second-order convergence, 而 Simpson's rule 通常表现为 fourth-order convergence。因为 h 和 $1/N$ 成正比, 所以误差大致按 N^{-2} 和 N^{-4} 下降。

Why can Simpson's rule be exact for a quadratic function? / 为什么 Simpson's rule 对二次函数可能直接精确?

Simpson's rule uses a quadratic polynomial to approximate the function over pairs of sub-intervals. Therefore, it can integrate polynomial functions up to degree 3 exactly, apart from round-off error. For a simple quadratic function such as $f(x) = x^2 + 1$, Simpson's rule may give the exact analytical integral even with a small number of sub-intervals.

Simpson's rule 用 quadratic polynomial 来近似每两个 sub-intervals 上的函数。因此它可以对 degree 3 及以下的 polynomial functions 精确积分, 只剩 round-off error。对于 $f(x) = x^2 + 1$ 这种简单二次函数, Simpson's rule 可能用很少的 sub-intervals 就得到 exact analytical integral。

Why is a sharply peaked function difficult to integrate numerically? / 为什么 sharp peak function 难以数值积分?

A sharply peaked function changes very rapidly in a small region but is relatively flat elsewhere. If a uniform grid uses too few sub-intervals, it may not place enough points near the peak, so the important contribution to the integral can be under-sampled. This makes simple quadrature rules such as the trapezoid rule less accurate unless the mesh is sufficiently refined.

Sharp peak function 会在一个很小区域内快速变化，但在其他区域相对平缓。如果 uniform grid 的 sub-intervals 太少，可能在 peak 附近没有足够的 points，因此积分中重要的贡献会被 under-sampled。这会让 trapezoid rule 等简单 quadrature rules 不够准确，除非 mesh 足够细。

Why does behaviour near $x = 0$ matter for this integration problem? / 为什么 $x = 0$ 附近的变化对积分很重要？

For a function such as $f(x) = 1 + 0.2 / (1 + 100x^2)$, the denominator is smallest near $x = 0$, so the function has a narrow central peak there. Away from $x = 0$, the function varies much more slowly. Therefore, the accuracy of the integral depends strongly on how well the numerical grid resolves the central peak.

对于 $f(x) = 1 + 0.2 / (1 + 100x^2)$ 这样的函数，denominator 在 $x = 0$ 附近最小，所以函数在那里有一个 narrow central peak。远离 $x = 0$ 时，函数变化慢得多。因此，积分精度强烈依赖 numerical grid 是否能充分 resolve 这个 central peak。

Why can even and odd numbers of sub-intervals give different integration errors? / 为什么偶数和奇数 sub-intervals 可能给出不同误差？

On a symmetric interval such as $[-1, 1]$, whether the grid includes the point $x = 0$ can strongly affect the result for a function with a sharp central peak. Some choices of N place a node exactly at the peak, while others place the peak between nodes. This can cause a noticeable even-odd difference in the trapezoid rule error.

在 $[-1, 1]$ 这样的 symmetric interval 上，对于有 sharp central peak 的函数，grid 是否包含 $x = 0$ 会明显影响结果。有些 N 会让 node 正好落在 peak 上，有些则让 peak 位于两个 nodes 之间。这会导致 trapezoid rule 的误差出现明显 even-odd difference。

Why can uniform discretisation be inefficient for rapidly varying functions? / 为什么 uniform discretisation 对快速变化函数效率低？

Uniform discretisation uses the same sub-interval width everywhere. For a function that varies rapidly only in a small region, many points are wasted in smooth regions, while the rapidly varying region may still be under-resolved. This means the method may require a large total N to achieve good accuracy.

Uniform discretisation 在整个区间内使用相同的 sub-interval width。对于只在小区内快速变化的函数，很多 points 会浪费在 smooth regions，而快速变化区域仍可能 under-resolved。这意味着方法可能需要很大的总 N 才能达到较好精度。

How can non-uniform discretisation improve integration accuracy? / non-uniform discretisation 如何提高积分精度?

Non-uniform discretisation places smaller sub-intervals where the function varies rapidly and larger sub-intervals where the function is smooth. For a peaked function, this means using more points near $x = 0$ and fewer points near $x = \pm 1$. This can improve the accuracy for the same number of function evaluations.

Non-uniform discretisation 会在函数快速变化的位置放更小的 sub-intervals, 在函数平缓的位置放更大的 sub-intervals。对于 peaked function, 这意味着在 $x = 0$ 附近放更多 points, 而在 $x = \pm 1$ 附近放更少 points。这样可以在相同 function evaluations 数量下提高 accuracy。

How should we place more points where the function varies rapidly? / 如何在函数变化快的位置放更多 points?

A good non-uniform grid should place more nodes in regions where the function has large gradients or strong curvature. For the peaked function, this means clustering points near $x = 0$ and using wider spacing away from the centre. The goal is to resolve the important part of the function without wasting too many points in smooth regions.

好的 non-uniform grid 应该在 function gradient 大或 curvature 强的位置放更多 nodes。对于 peaked function, 这意味着在 $x = 0$ 附近加密 points, 而在远离中心的位置使用更大的 spacing。目标是充分 resolve 函数最重要的部分, 同时避免在 smooth regions 浪费太多 points。

What is Weddle's rule? / Weddle's rule 是什么?

Weddle's rule is a higher-order Newton-Cotes quadrature rule used to approximate definite integrals. It uses several equally spaced points over a group of sub-intervals, so it can be more accurate than lower-order rules for smooth functions. Because it uses more points, the number of sub-intervals usually needs to be compatible with the rule.

Weddle's rule 是一种 higher-order Newton-Cotes quadrature rule, 用来近似 definite integrals。它会在一组 sub-intervals 上使用多个 equally spaced points, 所以对 smooth functions 可能比低阶规则更准确。因为它需要多个点, sub-intervals 的数量通常需要和这个 rule 兼容。

Why use one integration function with a rule argument? / 为什么用一个带 rule 参数的 integration function?

Using one integration function with a `rule` argument makes the code cleaner and easier to compare. The same function can call the Midpoint, Trapezoid, Simpson, or Weddle rule depending on the user's choice. This avoids repeated code and ensures that all methods are tested on the same interval, function, and number of sub-intervals.

使用一个带 `rule` 参数的 integration function 可以让代码更清晰，也更容易比较。根据用户选择，同一个 function 可以调用 Midpoint、Trapezoid、Simpson 或 Weddle rule。这样可以避免重复代码，并保证所有方法都在相同 interval、function 和 sub-interval number 下测试。

How can convergence analysis check for errors in quadrature code? / convergence analysis 如何检查 quadrature code 是否有错?

Convergence analysis checks whether the numerical error decreases as the number of sub-intervals increases. If an implementation is correct, the error should generally reduce at a rate consistent with the theoretical order of the quadrature rule. If the error does not decrease, or behaves irregularly, this may indicate a coding mistake or an incompatibility between the rule and the chosen `N`.

Convergence analysis 会检查 numerical error 是否随着 sub-intervals 数量增加而下降。如果 implementation 正确，error 通常应按 quadrature rule 的理论阶数下降。如果 error 不下降，或变化很不规律，可能说明有 coding mistake，或者 rule 和所选 `N` 不兼容。

Why compare all quadrature rules on the same integral? / 为什么要在同一个 integral 上比较所有 quadrature rules?

Comparing all quadrature rules on the same integral gives a fair test of their relative accuracy. The function, interval, and exact value are fixed, so differences in error mainly come from the numerical method itself. This makes it easier to identify which rule performs best for this particular smooth function.

在同一个 integral 上比较所有 quadrature rules，可以公平测试它们的 relative accuracy。Function、interval 和 exact value 都固定，所以 error 差异主要来自 numerical method 本身。这样更容易判断哪种 rule 对这个 smooth function 表现最好。

How can we verify quadrature code when the exact integral is unknown? / exact integral 不知道时如何验证 quadrature code?

If the exact integral is not obvious, we can compute a very accurate reference value using a trusted quadrature method with a very fine discretisation. This reference value is then treated as the “truth” for error calculations. The estimate can be trusted if further mesh refinement changes the result only by a very small amount.

如果 exact integral 不容易算出来，可以用 trusted quadrature method 和 very fine discretisation 计算一个高精度 reference value。这个 reference value 就可以暂时当作 “truth” 来计算 errors。如果继续细化 mesh 后结果只发生很小变化，就说明这个 estimate 比较可信。

How do we justify that a numerical integral estimate is accurate? / 如何说明 numerical integral estimate 足够准确?

A numerical integral estimate can be justified by showing that it is stable under further refinement. For example, if doubling the number of sub-intervals changes the result by a negligible amount, then the estimate is likely close to the true value. Using a high-order or trusted built-in quadrature routine can also support the accuracy claim.

可以通过展示结果在进一步 refinement 下很稳定, 来说明 numerical integral estimate 足够准确。例如, 如果把 sub-intervals 数量加倍后, 结果变化非常小, 那么这个 estimate 很可能接近 true value。使用 high-order 或 trusted built-in quadrature routine 也可以支持 accuracy claim。

Why can a trusted numerical estimate be used as “truth”? / 为什么 trusted numerical estimate 可以当作 “truth”?

In convergence testing, the true analytical integral is sometimes unavailable. A highly accurate numerical estimate can be used as a practical substitute for the truth, as long as it is much more accurate than the methods being tested. This allows approximate errors to be computed and convergence behaviour to be assessed.

在 convergence testing 中, 有时没有 true analytical integral。只要一个 high-accuracy numerical estimate 比正在测试的方法精确得多, 就可以把它作为 practical truth 的替代。这样就能计算 approximate errors, 并评估 convergence behaviour。

How is the composite trapezoidal rule derived? / composite trapezoidal rule 如何推导?

The composite trapezoidal rule divides the whole interval into many smaller sub-intervals and applies the trapezoid rule on each sub-interval. Adjacent trapezoids share interior nodes, so those function values should only be evaluated once. The final formula gives half weight to the two endpoints and full weight to the shared interior points.

Composite trapezoidal rule 把整个 interval 分成多个 smaller sub-intervals, 并在每个 sub-interval 上使用 trapezoid rule。相邻 trapezoids 会共享 interior nodes, 所以这些 function values 应该只计算一次。最终公式对两个 endpoints 给 half weight, 对 shared interior points 给 full weight。

Why does the composite trapezoidal rule reduce redundant function evaluations? / 为什么 composite trapezoidal rule 能减少重复函数计算?

In the composite trapezoidal rule, each interior node belongs to two neighbouring sub-intervals. Instead of evaluating the same function value twice, we store or reuse it once and give it the

correct combined weight. This makes the implementation more efficient while giving the same mathematical result.

在 composite trapezoidal rule 中, 每个 interior node 同时属于两个相邻 sub-intervals。我们不需要把同一个 function value 计算两次, 而是存储或重复使用一次, 并给它正确的 combined weight。这样实现更高效, 同时数学结果不变。

Why does Richardson extrapolation improve the trapezoidal rule? / 为什么 Richardson extrapolation 能改进 trapezoidal rule?

The trapezoidal rule has leading error proportional to h^2 . If I_T is the trapezoidal result with step size h , and I_{T2} is the result with half the step size, then the combination $I = (4I_{T2} - I_T) / 3$ cancels the leading h^2 error term. The new estimate is therefore more accurate than either trapezoidal result alone.

Trapezoidal rule 的 leading error 和 h^2 成正比。如果 I_T 是 step size 为 h 的 trapezoidal result, I_{T2} 是 step size 减半后的结果, 那么组合 $I = (4I_{T2} - I_T) / 3$ 可以抵消 leading h^2 error term。因此新估计比任意一个 trapezoidal result 单独使用都更准确。

Why is the Richardson-improved trapezoidal rule equivalent to Simpson's rule? / 为什么 Richardson 改进后的 trapezoidal rule 等价于 Simpson's rule?

Simpson's rule can be derived by combining two trapezoidal approximations with different step sizes. The formula $I = (4I_{T2} - I_T) / 3$ removes the leading second-order trapezoidal error and produces the same weighted combination of endpoint and midpoint values as Simpson's rule. Therefore, this Richardson extrapolation form is mathematically equivalent to Simpson's rule.

Simpson's rule 可以通过组合两个不同 step size 的 trapezoidal approximations 推导出来。公式 $I = (4I_{T2} - I_T) / 3$ 去掉了 trapezoidal rule 的 leading second-order error, 并产生和 Simpson's rule 相同的 endpoint 与 midpoint 加权形式。因此, 这个 Richardson extrapolation 形式在数学上等价于 Simpson's rule。

How can $\exp(x)$ be used to verify the trapezoidal rule? / 如何用 $\exp(x)$ 验证 trapezoidal rule?

A useful verification problem is $\int_0^1 \exp(x) dx = \exp(1) - 1$, because the exact answer is known. After implementing the composite trapezoidal rule, compute the integral with increasing numbers of sub-intervals and compare the numerical results with $\exp(1) - 1$. The error should decrease as the mesh is refined.

一个有用的 verification problem 是 $\int_0^1 \exp(x) dx = \exp(1) - 1$ ，因为 exact answer 已知。实现 composite trapezoidal rule 后，用不断增加的 sub-intervals 计算积分，并把 numerical results 和 $\exp(1) - 1$ 比较。随着 mesh refinement, error 应该下降。

When should we use Midpoint, Trapezoid, Simpson and Weddle's rules? / Midpoint、Trapezoid、Simpson 和 Weddle's rules 分别什么时候用?

Midpoint and Trapezoid rules are simple and easy to implement, so they are useful for basic quadrature and convergence testing. Simpson's rule is preferred for smooth functions because it is usually more accurate and is exact for polynomials up to degree 3. Weddle's rule is a higher-order rule for smooth functions, but it needs a compatible number of equally spaced sub-intervals.

Midpoint 和 Trapezoid rules 简单、容易实现，适合 basic quadrature 和 convergence testing。Simpson's rule 更适合 smooth functions，因为它通常更准确，并且对 degree 3 及以下 polynomial exact。Weddle's rule 是 higher-order rule，适合 smooth functions，但需要 sub-interval number 和 rule 兼容，并且通常要求 equally spaced points。

What are the limitations of Midpoint and Trapezoid rules? / Midpoint 和 Trapezoid rules 有什么限制?

Midpoint and Trapezoid rules are only second-order accurate for smooth functions, so their errors may decrease relatively slowly compared with higher-order methods. They can be inefficient for functions with sharp peaks or rapid changes, because a uniform grid may miss the important variation unless many sub-intervals are used.

Midpoint 和 Trapezoid rules 对 smooth functions 通常只是 second-order accurate，所以相比 higher-order methods，误差下降可能较慢。对于 sharp peaks 或 rapid changes 的函数，它们可能效率较低，因为 uniform grid 可能漏掉重要变化，除非使用很多 sub-intervals。

What are the limitations of Simpson's rule? / Simpson's rule 有什么限制?

Simpson's rule is more accurate for smooth functions, but it normally requires an even number of sub-intervals because it works over pairs of sub-intervals. It is less suitable if the data are not equally spaced, if the function is not smooth, or if the grid does not resolve sharp local variation. In those cases, non-uniform or adaptive methods may be better.

Simpson's rule 对 smooth functions 更准确，但通常要求 sub-intervals 数量为 even，因为它每次作用在一对子区间上。如果 data 不是 equally spaced，function 不够 smooth，或者 grid 没有 resolve sharp local variation，它就不一定合适。这些情况下 non-uniform 或 adaptive methods 可能更好。

What are the limitations of Weddle’s rule? / Weddle’s rule 有什么限制?

Weddle’s rule is a higher-order Newton–Cotes rule, so it can be very accurate for smooth functions. However, it requires a specific grouping of equally spaced points, so the number of sub-intervals must be compatible with the rule. It is not ideal for unevenly spaced data or functions with strong local peaks unless the grid is carefully chosen.

Weddle’s rule 是 higher-order Newton–Cotes rule，所以对 smooth functions 可以很准确。但它需要特定数量的 equally spaced points 组合，因此 sub-interval number 必须和 rule 兼容。对于 unevenly spaced data 或有 strong local peaks 的函数，如果 grid 没有 carefully chosen，就不太理想。

What is the rule applicability of numerical integration methods? / numerical integration rules 的适用性是什么?

The basic applicability rule is: use simple rules such as Midpoint or Trapezoid for robust first tests, use Simpson or Weddle for smooth functions with compatible uniform grids, and use non-uniform or adaptive discretisation when the function varies rapidly in a small region. The best rule depends on smoothness, grid spacing, and the number of available function evaluations.

基本适用原则是：先用 Midpoint 或 Trapezoid 做简单稳定的初步测试；对于 smooth functions 和兼容的 uniform grids，可以用 Simpson 或 Weddle；如果函数只在小区域 rapid variation，就用 non-uniform 或 adaptive discretisation。最佳 rule 取决于 smoothness、grid spacing 和可用的 function evaluations 数量。

Why is rule applicability important in quadrature? / 为什么 quadrature 中 rule applicability 很重要?

A quadrature rule can be mathematically correct but still perform poorly if its assumptions are not suitable for the problem. For example, Simpson’s rule is powerful for smooth uniformly spaced data, but it may not help if the function has a narrow peak that the grid misses. Therefore, choosing the rule should depend on the function shape and the data layout.

一个 quadrature rule 在数学上可能是正确的，但如果它的 assumptions 不适合当前问题，表现仍然可能很差。例如 Simpson’s rule 对 smooth、uniformly spaced data 很强，但如果函数有 narrow peak 且 grid 没有采到，它也不一定有效。因此选择 rule 时要看 function shape 和 data layout。

Chapter 4: ODE Solvers 常微分方程数值解

Method	Ratio	Order

FE	≈ 2.0	1st $O(\Delta t)$
BE	≈ 2.0	1st $O(\Delta t)$
Heun	≈ 4.0	2nd $O(\Delta t^2)$
IT	≈ 4.0	2nd $O(\Delta t^2)$

Why is the Trapezoidal method closer to the exact solution? / 为什么 Trapezoidal method 更接近精确解?

Forward Euler only uses the slope at the beginning of the step, so it may underestimate or overestimate the solution depending on the shape of the exact curve. Backward Euler uses information from the end of the step. The Trapezoidal method averages the beginning and ending slopes, so it usually gives a more balanced and more accurate approximation to the exact solution.

Forward Euler 只使用 step 起点处的 slope，所以可能根据 exact curve 的形状出现 underestimate 或 overestimate。Backward Euler 使用 step 终点的信息。Trapezoidal method 对起点和终点的 slopes 取平均，因此通常给出更平衡、更接近 exact solution 的近似。

What is the Forward Euler method for an ODE? / Forward Euler method 是什么?

Forward Euler is an explicit time-stepping method for solving an ODE such as $\frac{du}{dt} = f(u, t)$. It updates the solution using the slope at the current time level: $u_{n+1} = u_n + \Delta t f(u_n, t_n)$. Because all quantities on the right-hand side are already known, the method is simple to implement, but it is usually only first-order accurate.

Forward Euler 是一种 explicit time-stepping method，用来解类似 $\frac{du}{dt} = f(u, t)$ 的 ODE。它使用当前 time level 的 slope 更新解： $u_{n+1} = u_n + \Delta t f(u_n, t_n)$ 。因为右边所有量都是已知的，所以实现简单，但通常只有 first-order accuracy。

What is Heun's method or explicit trapezoidal method? / Heun's method 或 explicit trapezoidal method 是什么?

Heun's method first takes a Forward Euler prediction to estimate the solution at the next time level. It then averages the slope at the current point and the predicted next point. This is why it is also called the explicit trapezoidal method. It is usually more accurate than Forward Euler because it uses slope information from both ends of the time step.

Heun's method 先用 Forward Euler 做一个 prediction，估计下一个 time level 的解。然后它对当前点和预测点处的 slope 取平均。因此它也叫 explicit trapezoidal method。它通常比 Forward Euler 更准确，因为它使用了 time step 两端的 slope information。

Why should the exact solution not be used inside the ODE solver? / 为什么 exact solution 不能放进 ODE solver 里?

The exact solution should only be used for verification, not inside the numerical implementation. A solver should update the solution using only the ODE formula and previously computed numerical values. If the exact solution is used inside the code, the method is no longer being tested as a numerical solver, and the convergence analysis becomes meaningless.

Exact solution 只能用于 verification, 不能放进 numerical implementation 里。Solver 应该只使用 ODE formula 和之前计算出来的 numerical values 来更新解。如果在代码里直接使用 exact solution, 就不再是在测试 numerical solver, convergence analysis 也会失去意义。

How do we verify an ODE solver using convergence analysis? / 如何用 convergence analysis 验证 ODE solver?

To verify an ODE solver, solve the same problem using a sequence of decreasing time step sizes Δt . At the final time, compare the numerical solution with the exact solution and compute the error. If the solver is correct, the error should decrease at the expected rate as Δt becomes smaller.

验证 ODE solver 时, 可以用一系列不断减小的 time step sizes Δt 解同一个问题。在 final time, 把 numerical solution 和 exact solution 比较并计算 error。如果 solver 正确, 随着 Δt 变小, error 应该按预期 rate 下降。

Why does Backward Euler need rearrangement for this ODE? / 为什么 Backward Euler 在这道 ODE 里需要 rearrangement?

Backward Euler uses the slope at the new time level, so the unknown value u_{n+1} appears on both sides of the update formula. For the ODE $du/dt = u$, the formula is $u_{n+1} = u_n + \Delta t u_{n+1}$. This must be rearranged to $u_{n+1} = u_n / (1 - \Delta t)$ before it can be implemented directly.

Backward Euler 使用新 time level 的 slope, 所以 unknown value u_{n+1} 会同时出现在 update formula 的左右两边。对于 ODE $du/dt = u$, 公式是 $u_{n+1} = u_n + \Delta t u_{n+1}$ 。需要先整理成 $u_{n+1} = u_n / (1 - \Delta t)$, 才能直接实现。

What is the difference between explicit and implicit time-stepping? / explicit 和 implicit time-stepping 有什么区别?

In an explicit method, the new value is computed directly from known information at the current or previous time levels. In an implicit method, the new value appears inside the formula itself, so algebraic rearrangement or a solver may be needed. Explicit methods are simpler, while implicit methods can often be more stable.

在 explicit method 中, 新值可以直接由当前或之前 time levels 的已知信息计算出来。在 implicit method 中, 新值会出现在公式本身里面, 所以可能需要 algebraic rearrangement 或 solver。Explicit methods 更简单, 而 implicit methods 通常可能更 stable。

What is the implicit trapezoidal method? / implicit trapezoidal method 是什么?

The implicit trapezoidal method averages the slope at the current time level and the true new time level. For $\frac{du}{dt} = u$, it has the form $u_{n+1} = u_n + \Delta t/2 (u_n + u_{n+1})$. Unlike Heun's method, it does not use a predicted value. The new value must be solved for by rearranging the equation.

Implicit trapezoidal method 对当前 time level 和真实新 time level 的 slope 取平均。对于 $\frac{du}{dt} = u$, 它的形式是 $u_{n+1} = u_n + \Delta t/2 (u_n + u_{n+1})$ 。它不像 Heun's method 那样使用 predicted value, 而是需要通过 rearranging equation 解出新值。

How do we rearrange the implicit trapezoidal method for $\frac{du}{dt} = u$? / 如何整理 $\frac{du}{dt} = u$ 的 implicit trapezoidal method?

Starting from $u_{n+1} = u_n + \Delta t/2 (u_n + u_{n+1})$, collect the unknown terms on the left-hand side. This gives $u_{n+1}(1 - \Delta t/2) = u_n(1 + \Delta t/2)$. Therefore, the direct update formula is $u_{n+1} = u_n (1 + \Delta t/2) / (1 - \Delta t/2)$.

从 $u_{n+1} = u_n + \Delta t/2 (u_n + u_{n+1})$ 出发, 把 unknown terms 移到左边, 得到 $u_{n+1}(1 - \Delta t/2) = u_n(1 + \Delta t/2)$ 。因此直接更新公式是 $u_{n+1} = u_n (1 + \Delta t/2) / (1 - \Delta t/2)$ 。

Which ODE time-stepping method usually performs best here? / 这道 ODE 里哪个 time-stepping method 通常表现最好?

For the simple ODE $\frac{du}{dt} = u$, Forward Euler and Backward Euler are usually first-order accurate, while Heun's method and the implicit trapezoidal method are usually second-order accurate. Therefore, the trapezoidal-type methods normally give smaller errors for the same Δt . The best method should be identified by comparing errors in the convergence analysis.

对于简单 ODE $\frac{du}{dt} = u$, Forward Euler 和 Backward Euler 通常是 first-order accurate, 而 Heun's method 和 implicit trapezoidal method 通常是 second-order accurate。因此, 在相同 Δt

下, trapezoidal-type methods 通常会给出更小 errors。最终最好方法应通过 convergence analysis 的误差比较确定。

Why does Forward Euler underestimate or overestimate the exact solution? / 为什么 Forward Euler 可能低估或高估 exact solution?

Forward Euler uses only the slope at the beginning of each time step. If the true solution is curving upward, this starting slope is too small for the whole interval, so Forward Euler tends to underestimate the solution. For $dy/dt = y$ with $y(t) = \exp(t)$, the curve is increasing and convex, so Forward Euler usually lies below the exact solution.

Forward Euler 只使用每个 time step 起点处的 slope。如果 true solution 向上弯曲, 这个起点 slope 对整个 interval 来说偏小, 所以 Forward Euler 往往会 underestimate solution。对于 $dy/dt = y$ 和 $y(t) = \exp(t)$, 曲线 increasing 且 convex, 因此 Forward Euler 通常低于 exact solution。

Why does Backward Euler overestimate the exact solution for $\exp(t)$? / 为什么 Backward Euler 对 $\exp(t)$ 会高估?

Backward Euler uses the slope at the end of the time step. For an increasing convex solution such as $\exp(t)$, the end-point slope is larger than the average slope over the interval. Therefore, Backward Euler tends to step too far upward and overestimate the exact solution.

Backward Euler 使用 time step 终点处的 slope。对于 $\exp(t)$ 这种 increasing convex solution, 终点 slope 大于整个 interval 上的 average slope。因此 Backward Euler 往往向上走得太多, 从而 overestimate exact solution。

How can Forward and Backward Euler be combined to improve accuracy? / 如何结合 Forward 和 Backward Euler 提高精度?

Forward Euler may underestimate the solution, while Backward Euler may overestimate it for a convex increasing problem. A natural improvement is to average the two slopes, giving a trapezoidal-type method. This balances the beginning and ending slope information and often gives a more accurate approximation than either Euler method alone.

对于 convex increasing problem, Forward Euler 可能 underestimate, 而 Backward Euler 可能 overestimate。一个自然改进方法是平均两端 slopes, 得到 trapezoidal-type method。这样能平衡起点和终点的 slope information, 通常比单独使用任一 Euler method 更准确。

Why is the Trapezoidal method a combination of Forward and Backward Euler ideas? / 为什么 Trapezoidal method 可以看成结合了 Forward 和 Backward Euler?

The Trapezoidal method averages the slope used by Forward Euler and the slope used by Backward Euler. In ODE form, it uses $u_{n+1} = u_n + \Delta t/2 [f(u_n, t_n) + f(u_{n+1}, t_{n+1})]$. This is why it often lies between the Forward and Backward Euler solutions and gives better accuracy.

Trapezoidal method 平均了 Forward Euler 使用的 slope 和 Backward Euler 使用的 slope。在 ODE 形式中，它使用 $u_{n+1} = u_n + \Delta t/2 [f(u_n, t_n) + f(u_{n+1}, t_{n+1})]$ 。因此它通常位于 Forward 和 Backward Euler solutions 之间，并且精度更好。

Chapter 5: Root Finding 方程求根

How can Newton's method fail? / Newton 方法如何失效?

Newton's method can fail in several common ways. If $f'(x_n)$ is very close to zero, the Newton correction becomes very large, so the next iterate may jump far away and cause divergence or overflow. If the initial guess is poor, the method may also converge to a different root instead of the desired one. A practical remedy is to first bracket the root using bisection, then switch to Newton's method for rapid refinement.

Newton's method 有几种常见失效方式。如果 $f'(x_n)$ 很接近 0，Newton correction 会变得很大，导致下一步迭代跳得很远，从而引起 divergence 或 overflow。如果 initial guess 不够好，方法也可能收敛到另一个 root，而不是目标 root。一个常见的 fix 是先用 bisection bracket root，再切换到 Newton's method 进行快速 refinement。

How many real roots can a cubic have? / 三次函数可以有几个实根?

A cubic polynomial always has at least one real root and can have up to three real roots. By comparison, a linear function usually has 0 or 1 real root, and a quadratic has 0, 1, or 2 real roots. For a cubic, the exact number of real roots depends on the coefficients, or equivalently on the discriminant.

三次多项式一定至少有 1 个 real root，最多可以有 3 个 real roots。作为对比，linear function 通常有 0 或 1 个 real root，quadratic function 有 0、1 或 2 个 real roots。对于 cubic，实根的具体个数取决于 coefficients，也可以说取决于 discriminant。

What is Picard iteration? / 什么是 Picard iteration?

Picard iteration rewrites $f(x) = 0$ into the fixed-point form $x = g(x)$, then iterates using $x_{n+1} = g(x_n)$. It converges if $|g'(x)| < 1$ near the root, meaning the iteration moves points closer to the fixed point. The method is simple, but it may diverge if this slope condition is not satisfied.

Picard iteration 是把 $f(x) = 0$ 改写成 fixed-point form: $x = g(x)$, 然后用 $x_{n+1} = g(x_n)$ 迭代。如果在 root 附近满足 $|g'(x)| < 1$, 说明迭代会把点逐渐拉近 fixed point, 因此方法收敛。这个方法简单, 但如果不满足 slope condition, 就可能 diverge。

How do we locate roots from a plot and bracket them? / 如何通过图像估计 roots 并 bracket them?

To locate roots, first plot the function over the required interval and look for where the curve crosses the x -axis. A root can usually be bracketed by finding two points where the function changes sign. The interval is then repeatedly narrowed until the bracket size is below the required tolerance. The final result is a small interval containing the root, not necessarily a single exact value.

定位 roots 时, 先在给定区间画出 function, 看曲线在哪里穿过 x -axis。通常可以通过找到 function values 变号的两个点来 bracket root。之后不断缩小 interval, 直到 bracket size 小于要求的 tolerance。最终结果是一个包含 root 的小区间, 不一定是单个精确值。

How do Bisection, Secant, and Newton methods compare? / Bisection、Secant 和 Newton 方法如何比较?

Bisection is robust because it keeps the root bracketed, but it converges relatively slowly. The Secant method does not require the derivative and is usually faster than bisection, but convergence is not guaranteed. Newton's method is often fastest when the initial guess is good and the derivative is well behaved, but it can fail or converge to a different root when the starting point is poor.

Bisection 很 robust, 因为它始终保持 root 被 bracket 住, 但收敛相对较慢。Secant method 不需要 derivative, 通常比 bisection 更快, 但 convergence 不保证。Newton's method 在 initial guess 好且 derivative 表现正常时通常最快, 但如果 starting point 差, 可能 fail 或收敛到 different root。

Why can Newton's method converge to a different root? / 为什么 Newton's method 可能收敛到另一个 root?

Newton's method updates the estimate using the tangent line at the current point. If the initial guess is far from the intended root, or if the function has several roots, the tangent step may jump into the attraction region of another root. Therefore, Newton's method is a local method: it can converge quickly, but only when the starting guess is sufficiently close to the desired root.

Newton's method 每一步使用当前点的 tangent line 来更新估计值。如果 initial guess 离目标 root 太远，或者函数有多个 roots，切线步长可能跳到另一个 root 的吸引区域。因此 Newton's method 是 local method：它可以快速收敛，但前提是 starting guess 足够接近 desired root。

How can we demonstrate Newton failure using a chosen starting guess? / 如何用一个 starting guess 展示 Newton failure?

To demonstrate Newton failure, choose a starting guess where the derivative is very small, the tangent line is nearly horizontal, or the function has several nearby roots. Then record the Newton iterates and show that they jump far away, oscillate, diverge, or converge to a different root. The key is to explain why this starting guess leads to unexpected behaviour, not just to show the final answer.

展示 Newton failure 时，可以选择 derivative 很小、tangent line 接近水平，或者附近有多个 roots 的 starting guess。然后记录 Newton iterates，展示它们 jump far away、oscillate、diverge，或者收敛到 different root。关键是解释为什么这个 starting guess 会导致 unexpected behaviour，而不只是展示最终答案。

How many real roots can linear, quadratic and cubic functions have? / 一次、二次、三次函数分别可以有几个 real roots?

A linear function can have 0 or 1 real root if degenerate cases are allowed, for example a constant non-zero function has no root. A quadratic can have 0, 1, or 2 real roots depending on its discriminant. A true cubic with real coefficients has at least 1 real root and up to 3, but if degenerate cases are allowed, it can reduce to a quadratic, linear, or constant function.

如果允许 degenerate cases, linear function 可以有 0 或 1 个 real root, 例如非零常数函数没有 root。Quadratic 根据 discriminant 可以有 0、1 或 2 个 real roots。真正的 real-coefficient cubic 至少有 1 个 real root, 最多 3 个；但如果允许退化情况，它可以变成 quadratic、linear 或 constant function。

Why are linear and quadratic functions special cases of a cubic form? / 为什么 linear 和 quadratic 是 cubic form 的特殊情况?

The general cubic form is $f(x) = ax^3 + bx^2 + cx + d$. If $a = 0$, it reduces to a quadratic. If $a = 0$ and $b = 0$, it reduces to a linear function. If $a = b = c = 0$, it becomes a constant function. Therefore, root-count cases for lower-degree functions can be tested using the same general formula.

General cubic form 是 $f(x) = ax^3 + bx^2 + cx + d$ 。如果 $a = 0$ ，它退化为 quadratic。如果 $a = 0$ 且 $b = 0$ ，它退化为 linear function。如果 $a = b = c = 0$ ，它变成 constant function。因此，低阶函数的 root-count cases 可以用同一个 general formula 测试。

How can we scan an interval to locate all sign changes? / 如何扫描区间找到所有 sign changes?

To locate all possible roots, divide the search interval into many small sub-intervals and evaluate the function at each grid point. A sign change between two neighbouring points suggests that a root is bracketed in that sub-interval. Each detected bracket can then be refined using bisection or another root-finding method.

为了找到所有 possible roots，可以把搜索区间分成很多 small sub-intervals，并在每个 grid point 上计算 function value。相邻两点之间出现 sign change，说明这个小区间内可能 bracket 了一个 root。之后可以对每个 bracket 使用 bisection 或其他 root-finding method 进一步 refine。

How can bracketing and bisection be combined to find multiple roots? / 如何结合 bracketing 和 bisection 找多个 roots?

To find multiple roots, first scan the interval with many small sub-intervals and look for sign changes. Each sign-change interval becomes a bracket for a possible root. Then apply bisection separately to each bracket to refine the root location. Finally, plot the function and the computed roots to check whether the algorithm has found the expected roots.

找多个 roots 时，先用很多 small sub-intervals 扫描整个区间，寻找 sign changes。每一个 sign-change interval 都可以作为一个 possible root 的 bracket。然后对每个 bracket 分别使用 bisection 来 refine root location。最后画出 function 和 computed roots，检查 algorithm 是否找到了预期 roots。

Why should we test a root-finding code on 0, 1, 2 and 3 root cases? / 为什么 root-finding 代码要测试 0、1、2、3 roots 的情况?

Testing different root-count cases checks whether the algorithm is robust rather than only working for one example. A good root-finding code should correctly report no root when there is no sign change, one root when there is one crossing, and multiple roots when several brackets are detected. Plotting the result is a useful visual check.

测试不同 root-count cases 是为了检查 algorithm 是否 robust，而不是只对一个例子有效。好的 root-finding code 应该在没有 sign change 时正确报告 no root，在有一个 crossing 时找到 one root，在检测到多个 brackets 时找到 multiple roots。画图是很有用的 visual check。

Why can a bracketing algorithm miss repeated roots? / 为什么 bracketing algorithm 可能漏掉 repeated roots?

A bracketing algorithm based on sign change can miss repeated roots because the function may touch the x -axis and turn back without changing sign. In this case, scanning for $f(a)f(b) < 0$ will not detect the root. Extra checks, such as looking for very small $|f(x)|$ values or using derivative information, may be needed.

基于 sign change 的 bracketing algorithm 可能漏掉 repeated roots, 因为函数可能只是接触 x -axis 然后转回, 并没有变号。这种情况下, 扫描 $f(a)f(b) < 0$ 检测不到 root。可能需要额外检查, 例如寻找很小的 $|f(x)|$, 或者使用 derivative information。

How do we write a cubic function and its derivative for Newton's method? / 如何为 Newton's method 写 cubic function 和 derivative?

For a cubic function $f(x) = ax^3 + bx^2 + cx + d$, Newton's method also needs the derivative. The derivative is $f'(x) = 3ax^2 + 2bx + c$. Writing separate functions for $f(x)$ and $f'(x)$ makes it clear that a full Newton method is being used, because the derivative information is explicitly included.

对于 cubic function $f(x) = ax^3 + bx^2 + cx + d$, Newton's method 还需要 derivative。导数是 $f'(x) = 3ax^2 + 2bx + c$ 。分别写 $f(x)$ 和 $f'(x)$ 的 function, 可以清楚说明使用的是 full Newton method, 因为 derivative information 被明确使用了。

What is the difference between full Newton and quasi-Newton methods? / full Newton 和 quasi-Newton method 有什么区别?

A full Newton method uses the actual derivative $f'(x)$ in the update formula $x_{n+1} = x_n - f(x_n)/f'(x_n)$. A quasi-Newton method approximates derivative information instead of using the exact derivative directly. If the question asks for a full Newton method, the derivative function must be evaluated explicitly.

Full Newton method 在 update formula $x_{n+1} = x_n - f(x_n)/f'(x_n)$ 中使用真实 derivative $f'(x)$ 。Quasi-Newton method 则近似 derivative information, 而不是直接使用 exact derivative。如果题目要求 full Newton method, 就必须明确计算并调用 derivative function。

How can a cubic function demonstrate one-root and three-root cases? / cubic function 如何展示 one-root 和 three-root 情况?

Different choices of the parameters a , b , c , and d can give cubic functions with different numbers of real roots. A monotonic cubic may have only one visible crossing of the x -axis, while a cubic with two turning points may cross the axis three times. Plotting the function is a useful way to confirm the chosen root scenario before applying numerical methods.

不同的参数 `a`、`b`、`c` 和 `d` 可以产生不同 real roots 数量的 cubic functions。Monotonic cubic 可能只有一个明显的 `x`-axis crossing，而有两个 turning points 的 cubic 可能与 `x`-axis 相交三次。应用 numerical methods 前，画图可以帮助确认选择的 root scenario。

How can Newton's method cause overflow errors? / Newton's method 为什么可能导致 overflow errors?

Newton's method can produce overflow if the iterates jump to extremely large values. This may happen when the derivative is very small, so the correction $f(x_n)/f'(x_n)$ becomes huge, or when the starting guess sends the iteration far from the root. Once the numbers become too large for the computer representation, overflow occurs.

如果 Newton's method 的 iterates 跳到极大的数值，就可能产生 overflow。常见原因是 derivative 很小，使 correction $f(x_n)/f'(x_n)$ 变得巨大，或者 starting guess 让 iteration 远离 root。当数值大到超过计算机能表示的范围时，就会出现 overflow。

What is the quasi-Newton method in one-dimensional root finding? / 一维 root finding 中 quasi-Newton method 是什么?

A quasi-Newton method has a Newton-like update but does not use the exact derivative directly. Instead, it approximates derivative information, often using a finite difference slope. This can be useful when the derivative is unavailable or difficult to compute, but it may be less accurate or less reliable than full Newton's method with the exact derivative.

Quasi-Newton method 的 update 形式类似 Newton，但不直接使用 exact derivative。它通常用 finite difference slope 来近似 derivative information。当 derivative 不可用或难以计算时，这很有用，但它可能不如使用 exact derivative 的 full Newton's method 准确或稳定。

How do bisection, quasi-Newton and secant methods differ? / bisection、quasi-Newton 和 secant methods 有什么区别?

Bisection requires a bracket and is very robust, but it converges slowly. A quasi-Newton method uses an approximate derivative to make Newton-like updates, so it can be faster but less guaranteed. The secant method uses two previous points to approximate the derivative, so it avoids an exact derivative while often converging faster than bisection.

Bisection 需要一个 bracket，方法很 robust，但收敛较慢。Quasi-Newton method 使用 approximate derivative 做类似 Newton 的更新，所以可能更快，但不一定保证收敛。Secant method 用前两个点近似 derivative，因此不需要 exact derivative，同时通常比 bisection 收敛快。

How can the chain rule help construct an exact derivative for Newton's method? / chain rule 如何帮助构造 Newton's method 的 exact derivative?

If a function is made from terms such as $g(x) = 1 / ((x-a)^2 + b)$, the chain rule can be used to derive its exact derivative. For this term, the derivative is $g'(x) = -2(x-a) / ((x-a)^2 + b)^2$. Using the exact derivative allows a full Newton method to be applied rather than a quasi-Newton approximation.

如果函数由类似 $g(x) = 1 / ((x-a)^2 + b)$ 的项组成, 可以用 chain rule 推导 exact derivative。这个项的导数是 $g'(x) = -2(x-a) / ((x-a)^2 + b)^2$ 。使用 exact derivative 就可以应用 full Newton method, 而不是 quasi-Newton approximation。

Why can full Newton behave differently from quasi-Newton? / 为什么 full Newton 和 quasi-Newton 表现可能不同?

Full Newton uses the exact derivative, so its local slope information is more accurate. Quasi-Newton uses an approximate slope, which may introduce extra error and affect the update direction or step size. Therefore, full Newton can converge faster near a simple root, while quasi-Newton may be easier to implement when derivative information is unavailable.

Full Newton 使用 exact derivative, 所以 local slope information 更准确。Quasi-Newton 使用 approximate slope, 可能引入额外误差, 并影响 update direction 或 step size。因此, full Newton 在 simple root 附近可能收敛更快, 而 quasi-Newton 在 derivative information 不可用时更容易实现。

How can a quadratic function have zero, one or two real roots? / quadratic function 如何有 0、1 或 2 个 real roots?

A quadratic function $f(x) = ax^2 + bx + c$ can have different numbers of real roots depending on the discriminant $b^2 - 4ac$. If the discriminant is negative, there are no real roots. If it is zero, there is one repeated real root. If it is positive, there are two distinct real roots.

Quadratic function $f(x) = ax^2 + bx + c$ 的 real roots 数量取决于 discriminant $b^2 - 4ac$ 。如果 discriminant 小于 0, 没有 real roots。如果等于 0, 有一个 repeated real root。如果大于 0, 有两个不同的 real roots。

Why is a repeated quadratic root difficult for Newton's method? / 为什么 repeated quadratic root 对 Newton's method 更难?

At a repeated root, the curve touches the x -axis rather than crossing it, and the derivative near the root may be small. Newton's method can then converge more slowly than usual or behave unexpectedly from some starting guesses. This also makes sign-change bracketing harder, because the function may not change sign across the root.

在 repeated root 处，曲线只是接触 x -axis，而不是穿过它，并且 root 附近的 derivative 可能较小。因此 Newton's method 可能比正常情况收敛更慢，或在某些 starting guesses 下表现异常。这也让 sign-change bracketing 更难，因为函数在 root 两侧可能不变号。

Chapter 6: Linear Systems — Hand Calculation 线性系统

How do we choose a suitable 3 by 3 linear system? / 如何选择合适的 3 by 3 linear system?

A suitable 3 by 3 linear system should have a non-diagonal coefficient matrix, a non-zero right-hand-side vector, and a unique solution. A unique solution requires $\det(A) \neq 0$, meaning the matrix is invertible. Choosing a non-trivial system avoids cases where the solution is too obvious or where the equations are not useful for demonstrating elimination.

合适的 3 by 3 linear system 应该有 non-diagonal coefficient matrix、non-zero right-hand-side vector，并且有 unique solution。Unique solution 要求 $\det(A) \neq 0$ ，也就是 matrix invertible。选择 non-trivial system 可以避免答案过于明显，或方程不适合展示 elimination 的情况。

What does $\det(A) \neq 0$ tell us about a linear system? / $\det(A) \neq 0$ 对 linear system 说明什么?

If $\det(A) \neq 0$, the coefficient matrix is invertible and the linear system has a unique solution. If $\det(A) = 0$, the system may have no solution or infinitely many solutions, depending on the right-hand side. Therefore, checking the determinant is a quick way to avoid singular systems when choosing examples.

如果 $\det(A) \neq 0$ ，coefficient matrix 是 invertible，linear system 有 unique solution。如果 $\det(A) = 0$ ，系统可能没有解，也可能有 infinitely many solutions，取决于 right-hand side。因此，检查 determinant 是选择例题时避免 singular systems 的快速方法。

How do we transform a system into lower-triangular form? / 如何把系统变成 lower-triangular form?

To form a lower-triangular system, row operations are performed in the opposite direction from standard Gaussian elimination. Starting from the bottom row as the pivot row, eliminate entries above the pivot rather than below it. The result is a lower-triangular system, which can be solved from the top downward using forward substitution.

要得到 lower-triangular system，需要用和 standard Gaussian elimination 相反的方向进行 row operations。从 bottom row 作为 pivot row 开始，消去 pivot 上方的元素，而不是下方元素。最终得到 lower-triangular system，可以用 forward substitution 从上往下求解。

What is forward substitution? / 什么是 forward substitution?

Forward substitution solves a lower-triangular system starting from the first equation. The first row gives the first unknown directly, then this value is substituted into the next row to solve the next unknown. This continues downward until all unknowns have been found.

Forward substitution 是从第一条方程开始解 lower-triangular system。第一行可以直接求出第一个 unknown，然后把这个值代入下一行求下一个 unknown。这个过程不断向下进行，直到所有 unknowns 都求出。

How does lower-triangular elimination differ from standard Gaussian elimination? / lower-triangular elimination 和 standard Gaussian elimination 有什么区别?

Standard Gaussian elimination eliminates entries below the main diagonal and produces an upper-triangular system solved by back substitution. Lower-triangular elimination eliminates entries above the main diagonal and produces a lower-triangular system solved by forward substitution. Both methods use row operations and should give the same solution if no arithmetic mistakes are made.

Standard Gaussian elimination 消去 main diagonal 下方的元素，得到 upper-triangular system，并用 back substitution 求解。Lower-triangular elimination 消去 main diagonal 上方的元素，得到 lower-triangular system，并用 forward substitution 求解。两者都使用 row operations，如果没有算错，应该得到相同 solution。

What should a lower-triangular Python solver do? / lower-triangular Python solver 应该做什么?

A lower-triangular Python solver should first apply row operations to transform the augmented system $[A|b]$ into lower-triangular form. It should then use forward substitution to compute the unknowns. The code explanation should connect each loop to the mathematical operation: selecting pivots, eliminating entries above the pivot, updating the right-hand side, and solving unknowns in order.

一个 lower-triangular Python solver 应该先对 augmented system $[A|b]$ 做 row operations，把它变成 lower-triangular form。然后使用 forward substitution 计算 unknowns。代码解释需要把每个 loop 和数学操作对应起来：选择 pivots、消去 pivot 上方元素、更新 right-hand side、按顺序求 unknowns。

What are the two main steps of Gaussian Elimination? / Gaussian Elimination 的两个主要步骤是什么？

Gaussian Elimination has two main steps. First, row operations transform the augmented matrix $[A|b]$ into upper-triangular form. Second, back substitution solves the resulting triangular system from the last equation upward. These two steps correspond to elimination first and then solving the simplified system.

Gaussian Elimination 有两个主要步骤。第一步，用 row operations 把 augmented matrix $[A|b]$ 变成 upper-triangular form。第二步，用 back substitution 从最后一条方程向上求解 triangular system。这两个步骤对应先 elimination，再解简化后的系统。

How should we explain Gaussian Elimination mathematically and in code? / 如何同时解释 Gaussian Elimination 的数学和代码？

A good explanation should connect each mathematical operation to the corresponding line of code. The outer loop chooses the pivot column, the inner loop eliminates entries below the pivot, and the row update changes both the coefficient matrix and the right-hand-side vector. After the matrix is upper triangular, a second loop performs back substitution to compute the unknowns.

好的解释应该把每个 mathematical operation 和对应的 code line 联系起来。Outer loop 选择 pivot column，inner loop 消去 pivot 下方的 entries，row update 同时改变 coefficient matrix 和 right-hand-side vector。矩阵变成 upper triangular 后，第二个 loop 用 back substitution 计算 unknowns。

Why must row operations also update the right-hand side? / 为什么 row operations 也必须更新 right-hand side?

Row operations represent operations on the whole equation, not only on the coefficient matrix. Therefore, when a multiple of one row is subtracted from another row, the same operation must also be applied to the right-hand-side value. Otherwise, the transformed system would no longer be equivalent to the original system.

Row operations 表示对整条 equation 做操作，而不只是对 coefficient matrix 操作。因此，当用一行的倍数去减另一行时，同样的操作也必须作用在 right-hand-side value 上。否则，变换后的系统就不再和 original system 等价。

Why is a step-by-step code explanation important for Gaussian Elimination? / 为什么 Gaussian Elimination 需要 step-by-step code explanation?

Gaussian Elimination is not only a formula but an algorithm. A step-by-step explanation shows how the code selects pivots, computes multipliers, updates rows, creates zeros below the diagonal, and finally solves by back substitution. This demonstrates understanding of both the mathematics and the implementation.

Gaussian Elimination 不只是一个公式，而是一个 algorithm。Step-by-step explanation 可以展示代码如何选择 pivots、计算 multipliers、更新 rows、在 diagonal 下方制造 zeros，并最终用 back substitution 求解。这能同时体现对 mathematics 和 implementation 的理解。

How can row operations be used to compute an inverse matrix? / 如何用 row operations 求 inverse matrix?

To compute an inverse matrix, form the augmented system $[A | I]$, where I is the identity matrix. Then apply row operations to transform the left side A into I . If the operations are valid and A is invertible, the right side will become A^{-1} , giving $[I | A^{-1}]$.

求 inverse matrix 时，可以构造 augmented system $[A | I]$ ，其中 I 是 identity matrix。然后用 row operations 把左边的 A 变成 I 。如果操作合法且 A 是 invertible，右边就会变成 A^{-1} ，最终得到 $[I | A^{-1}]$ 。

Why does $[A | I]$ become $[I | A^{-1}]$? / 为什么 $[A | I]$ 会变成 $[I | A^{-1}]$?

Row operations are equivalent to multiplying by elementary matrices. If these operations transform A into I , then their combined effect is multiplication by A^{-1} . Applying the same operations to the identity matrix therefore produces A^{-1} . This is why reducing $[A | I]$ gives $[I | A^{-1}]$.

Row operations 等价于乘以 elementary matrices。如果这些操作把 A 变成 I ，那么它们的整体效果相当于乘以 A^{-1} 。同样的操作作用在 identity matrix 上，就会得到 A^{-1} 。这就是为什么化简 $[A | I]$ 会得到 $[I | A^{-1}]$ 。

What should matrix inverse code do with A and I ? / matrix inverse 代码应该如何处理 A 和 I ?

Matrix inverse code should apply the same row operations to both the matrix A and the identity matrix I . The goal is to transform A into the identity matrix. As this happens, the matrix that started as I is transformed into A^{-1} . The code does not necessarily need to form one large augmented matrix, but the operations must remain consistent.

Matrix inverse code 应该对 matrix A 和 identity matrix I 同时应用相同的 row operations。目标是把 A 变成 identity matrix。与此同时，原本是 I 的矩阵会被变成 A^{-1} 。代码不一定非要形成一个大的 augmented matrix，但两边的操作必须保持 consistent。

Why must A be invertible to compute A^{-1} ? / 为什么 A 必须 invertible 才能求 A^{-1} ?

A matrix must be invertible in order to have an inverse. If $\det(A) = 0$, the matrix is singular, so row operations cannot reduce it fully to the identity matrix. In that case, A^{-1} does not exist, and inverse-based methods cannot be used.

Matrix 必须 invertible 才有 inverse。如果 $\det(A) = 0$ ，matrix 是 singular，因此 row operations 不能把它完全化成 identity matrix。这种情况下， A^{-1} 不存在，也不能使用基于 inverse 的方法。

What does LU decomposition without pivoting mean? / LU decomposition without pivoting 是什么意思?

LU decomposition without pivoting means factorising A into $A = LU$ using Gaussian elimination without swapping rows. The matrix L stores the elimination multipliers, and U is the resulting upper-triangular matrix. This is simpler for hand calculation, but it can fail or become unstable if a pivot is zero or very small.

LU decomposition without pivoting 是指在不交换 rows 的情况下，用 Gaussian elimination 把 A 分解成 $A = LU$ 。Matrix L 存储 elimination multipliers， U 是得到的 upper-triangular matrix。这种方法适合手算，但如果 pivot 为 0 或很小，可能 fail 或变得 unstable。

Why are row swaps avoided in no-pivoting LU derivation? / 为什么 no-pivoting LU 推导中避免 row swaps?

If row swaps are used, the decomposition becomes $PA = LU$, where P is a permutation matrix. When the question asks for no partial pivoting and no row swaps, it expects the simpler form $A = LU$. Therefore, the hand derivation should keep the original row order and store only the elimination multipliers in L .

如果使用 row swaps，分解形式会变成 $PA = LU$ ，其中 P 是 permutation matrix。当题目要求 no partial pivoting 和 no row swaps 时，它想要的是更简单的 $A = LU$ 。因此手推时应保持原始 row order，只把 elimination multipliers 存入 L 。

How does Gauss-Seidel solve a linear system iteratively? / Gauss-Seidel 如何迭代求解 linear system?

Gauss-Seidel solves a linear system by updating each unknown one at a time using the most recent available values. In each iteration, newly computed components are immediately used in the following component updates. The process continues until the residual or the change in the solution becomes smaller than a chosen tolerance.

Gauss-Seidel 通过逐个更新 unknowns 来解 linear system, 并且每次都使用当前最新的 values。在每一轮 iteration 中, 刚算出的 components 会立刻用于后续 components 的更新。这个过程持续到 residual 或 solution change 小于设定 tolerance。

How can Gauss-Seidel agreement with the direct solution become arbitrarily close? / Gauss-Seidel 如何任意接近 direct solution?

If Gauss-Seidel converges, its approximation can be made arbitrarily close to the direct solution by using a stricter tolerance or more iterations. The difference between the iterative and direct solutions should decrease as the stopping criterion becomes tighter. This demonstrates that the iterative method is approaching the same true solution.

如果 Gauss-Seidel 收敛, 通过使用更严格的 tolerance 或更多 iterations, 它的 approximation 可以任意接近 direct solution。随着 stopping criterion 变得更严格, iterative solution 和 direct solution 之间的差异应该减小。这说明 iterative method 正在接近同一个 true solution。

Why compare Gauss-Seidel with an LU or direct solution? / 为什么要把 Gauss-Seidel 和 LU/direct solution 比较?

Comparing Gauss-Seidel with an LU or direct solution checks whether the iterative method is converging to the correct answer. The direct method provides a reference solution up to round-off error. If the Gauss-Seidel result gets closer to this reference as iterations increase, the implementation and convergence behaviour are likely correct.

把 Gauss-Seidel 和 LU 或 direct solution 比较, 可以检查 iterative method 是否收敛到正确答案。Direct method 提供一个只受 round-off error 影响的 reference solution。如果 Gauss-Seidel result 随 iterations 增加逐渐接近这个 reference, 说明 implementation 和 convergence behaviour 大概率正确。

Chapter 7: Iterative Methods & Condition Number 迭代法与条件数

Direct vs Iterative Methods / 直接法 vs 迭代法

Direct methods solve a linear system in a finite number of algebraic operations, giving the exact solution up to round-off error. Iterative methods generate a sequence of approximations that should converge toward the true solution. Their final accuracy is usually controlled by a residual tolerance or stopping criterion. Gauss-Seidel is generally preferred over Jacobi because it uses updated values immediately.

Direct methods 通过有限次 algebraic operations 解 linear system, 得到的解主要受 round-off error 影响。Iterative methods 生成一系列 approximations, 并逐步收敛到 true solution。最终精度通常由 residual tolerance 或 stopping criterion 控制。Gauss-Seidel 通常优于 Jacobi, 因为它会立即使用更新后的 values。

Why does Gauss-Seidel converge faster than Jacobi? / 为什么 Gauss-Seidel 比 Jacobi 收敛更快?

Gauss-Seidel usually converges faster than Jacobi because it immediately uses the most recently updated component values within the same iteration. Jacobi waits until the next iteration before using the newly computed values. Therefore, Gauss-Seidel often makes more efficient progress toward the solution.

Gauss-Seidel 通常比 Jacobi 收敛更快, 因为它会在同一轮 iteration 中立即使用最新更新的 component values。Jacobi 则要等到下一轮 iteration 才使用刚算出的新值。因此, Gauss-Seidel 通常能更高效地接近 solution。

What is the condition number? / 什么是条件数?

The condition number is defined as $\kappa(A) = \|A\| \|A^{-1}\|$. If $\kappa(A) \approx 1$, the matrix is well-conditioned. If $\kappa(A) \gg 1$, the matrix is ill-conditioned, meaning tiny round-off errors or small changes in the data can cause very large changes in the solution x . Common examples include the Hilbert matrix and the Vandermonde matrix.

Condition number 定义为 $\kappa(A) = \|A\| \|A^{-1}\|$ 。如果 $\kappa(A) \approx 1$, 矩阵是 well-conditioned。如果 $\kappa(A) \gg 1$, 矩阵是 ill-conditioned, 说明很小的 round-off errors 或 data perturbations 都可能导致解 x 发生很大变化。常见例子包括 Hilbert matrix 和 Vandermonde matrix。

What is the diagonal dominance test? / 什么是对角占优测试?

A matrix is strictly diagonally dominant if the absolute value of each diagonal entry is greater than the sum of the absolute values of the other entries in the same row. In symbols, this means $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for every row. If a matrix is strictly diagonally dominant, both Jacobi and Gauss-Seidel methods are guaranteed to converge.

如果每一行中 diagonal entry 的绝对值都大于该行其他元素绝对值之和, 则矩阵是 strictly diagonally dominant。也就是每一行都满足 $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ 。如果矩阵严格对角占优, 那么 Jacobi 和 Gauss-Seidel methods 都保证收敛。